Introduction to Robotics

Extended course notes, built from first principles

Mukhammadrizo

Graduate Introduction to Robotics

Compiled November 3, 2026

Contents

Conventions and notation ii

1 Coordinate Frames and Planar Rigid-Body Motion 1

1.1 Coordinate Frames and Relative Pose . . . 1
1.2 Rotation in 2D . . . 1
1.3 Pose and Homogeneous Transformations . . . 1
1.4 Pose Graphs . . . 1
1.5 Matrix Exponentials and Twists . . . 1

2 Orientation, Rotations and Rigid-Body Motion 11

2.1 Prerequisites: the language of 3-D rotation . . . 11
2.2 Orientation in 3-D . . . 13
2.3 Three angles and gimbal lock . . . 16
2.4 Angle–axis and exponential coordinates . . . 19
2.5 Unit quaternions . . . 23
2.6 3-D pose and screw motion . . . 26
2.7 Putting it together . . . 31

Conventions and notation

These conventions hold in every chapter. When a chapter needs new notation, it is added here first, so the whole book stays consistent. Most confusion in robotics comes from mismatched conventions, not from hard mathematics.

Frames and coordinates

- Frames are written $\{A\},\{B\},\ldots$; reserved names: $\{W\}$ world, $\{B\}$ robot base/body, $\{C\}$ camera, $\{I\}$ IMU, $\{E\}$ end-effector. ${}^{A}p$ = coordinates of point/vector p expressed in frame $\{A\}$.
- $R_{AB} \in SO(3)$ is the orientation of $\{B\}$ *relative to* $\{A\}$. As a matrix it converts coordinates *from B to A*: ${}^{A}p = R_{AB}\,{}^{B}p$. Inner indices cancel: $R_{AC} = R_{AB}R_{BC}$.
- Poses: $T_{AB} \in SE(3)$, with t_{AB} the origin of $\{B\}$ written in $\{A\}$; ${}^{A}p = R_{AB}\,{}^{B}p + t_{AB}$.
- ROS body convention: x forward, y left, z up. Camera optical convention: z forward, x right, y down.

Rotations and their parameterisations

- $[\omega]_\times$: the 3×3 skew matrix with $[\omega]_\times v = \omega \times v$. $\mathrm{Exp} : \mathbb{R}^3 \to SO(3)$ and $\mathrm{Log} : SO(3) \to \mathbb{R}^3$ map between rotation vectors and matrices; $e^{(\cdot)}$ is the ordinary matrix exponential.
- Euler angles: aerospace/ROS "RPY" $R = R_z(\psi)R_y(\theta)R_x(\phi)$: yaw ψ, pitch θ, roll ϕ.
- Quaternions: Hamilton ($ij = k$), scalar first, $q = (w,x,y,z) = (w,u)$, product $\otimes$. ROS messages, Eigen's storage and SciPy output use (x,y,z,w).
- Angular velocity: $\dot{R} = [\omega_s]_\times R = R[\omega_b]_\times$ (s = spatial/world, b = body).

Planar motion (chapter 1)

- $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ is the planar rotation generator ($J^2 = -I$; Jp is p turned by 90°). $R(\theta) = e^{\theta J} \in SO(2)$. (Chapter 2 uses J for Jacobians; the context always makes clear which is meant.)
- Planar poses $T = \begin{bmatrix} R(\theta) & t \\ 0 & 1 \end{bmatrix} \in SE(2)$. Planar twists are linear-first like 3-D ones: $S = (v_x, v_y, \omega)$, $[S] = \begin{bmatrix} \omega J & v \\ 0 & 0 \end{bmatrix} \in \mathfrak{se}(2)$; vexa is the inverse of $[\cdot]$. A unit twist $\hat{S}$ is scaled by a motion amount λ (an angle or a distance): $T = \exp(\lambda[\hat{S}])$.

Rigid-body motion

- Twists are ordered linear-first, $\xi = (v,\omega) \in \mathbb{R}^6$, as in Murray–Li–Sastry, Barfoot and Sophus. Lynch & Park (*Modern Robotics*) and GTSAM use (ω, v); the adjoint blocks swap accordingly.
- $[\xi] = \begin{bmatrix} [\omega]_\times & v \\ 0 & 0 \end{bmatrix} \in \mathfrak{se}(3)$; $\mathrm{Ad}_T = \begin{bmatrix} R & [t]_\times R \\ 0 & R \end{bmatrix}$.
- Perturbations are on the right (body frame) unless stated: $R\,\mathrm{Exp}(\delta)$, $T\,\mathrm{Exp}(\delta)$.

General

- Vectors are columns. $c_\theta = \cos\theta$, $s_\theta = \sin\theta$. Angles in radians in formulas, degrees in examples.
- Box types: **Key idea** (the essence), **Derivation** (full argument), **Worked example** (applied, numbers verified in code), **Pitfall** (what goes wrong in practice), plus plain note boxes.

Chapter 1

Coordinate Frames and Planar Rigid-Body Motion

This chapter builds the language of robot pose in the plane: frames and relative poses, planar rotations, homogeneous transformations, pose graphs, and finally the matrix exponential and twists, which describe how a pose *moves*. Everything here is the two-dimensional special case of chapter 2. Because the group is small enough to write out by hand, every idea can be seen in full.

1.1 Coordinate Frames and Relative Pose

Not yet written.

1.2 Rotation in 2D

Not yet written.

1.3 Pose and Homogeneous Transformations

Not yet written.

1.4 Pose Graphs

Not yet written.

1.5 Matrix Exponentials and Twists

So far a pose $T \in SE(2)$ has been a static object: where the robot *is*. A controller, an odometry filter or a motion planner needs the dynamic counterpart: a description of how the pose *changes* that can be scaled, integrated, compared and inverted. The naive choice, "add a small $(\Delta x, \Delta y, \Delta\theta)$ to the pose", fails for the same reason it fails in 3-D: poses do not live in a vector space, and a robot that turns while it drives does not move along a straight line. This section builds the right object from one physical picture: a robot repeating the same tiny body-frame step over and over.

Key idea

A constant body-frame motion is generated by one matrix $[S] \in \mathfrak{se}(2)$, the *twist*: it holds the forward velocity and the turn rate per unit of "motion amount" λ. Repeating an infinitesimal step $I + \delta\lambda\,[S]$ infinitely often gives the exact finite motion $T(\lambda) = \exp(\lambda[S])$. In the plane this is always a rotation about a single point, the turn centre $C = Jv/\omega$, or a pure translation. The logarithm goes back from a pose to the twist, but only up to a choice of branch. It recovers the *shortest* generator, not the motion history.

1.5.1 One small step: move, then turn

Body-frame increments compose on the right. Let $T_k = T_{WB_k}$ be the robot's pose at step k. Over the next short interval the robot moves by a displacement $\Delta T = T_{B_kB_{k+1}}$ that is known *in its own body frame*: wheel encoders, an IMU or a motion command never know where the world origin is. Index cancellation (§2.6.2) fixes where ΔT goes:

$$T_{k+1} = T_{WB_{k+1}} = T_{WB_k}\,T_{B_kB_{k+1}} = T_k\,\Delta T. \tag{1.1}$$

Read it as a map of points: a point with coordinates p in the new body frame has coordinates $\Delta T\, p$ in the old body frame, and $T_k \Delta T\, p$ in the world. A body-frame increment always goes on the *right*. Putting it on the left, $\Delta T\, T_k$, would apply the same numbers as a motion about the *world* origin along *world* axes. That is a different motion as soon as the robot is not at the origin facing $+x$.

The step as a matrix. Write $T_x(d)$ for a pure translation by d along the current body x-axis and $T_R(\alpha)$ for a pure rotation by α about the current body origin. A short arc of radius r that turns the heading by $\delta\theta$ has length $\delta s = r\,\delta\theta$. The simplest model of it is "drive straight δs, then turn in place by $\delta\theta$":

$$\Delta T = T_x(\delta s)\, T_R(\delta\theta) = \begin{bmatrix} 1 & 0 & r\,\delta\theta \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\delta\theta & -\sin\delta\theta & 0 \\ \sin\delta\theta & \cos\delta\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos\delta\theta & -\sin\delta\theta & r\,\delta\theta \\ \sin\delta\theta & \cos\delta\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The order matters. Read as a chain of frames, $T_x(\delta s)\, T_R(\delta\theta) = T_{B_k B'}\, T_{B' B_{k+1}}$: the left factor is the *first* motion (drive to an intermediate frame B'), and the right factor is expressed in B' (pivot about the new position). Pivoting does not move the origin, so the translation column stays $(r\,\delta\theta, 0)$. For $T_R(\delta\theta)\, T_x(\delta s)$ (pivot, then drive along the *new* heading) it would be $r\,\delta\theta(\cos\delta\theta, \sin\delta\theta)$.

Derivation: how far is "move, then turn" from the real arc?

On the real arc the robot turns about the centre $C = (0, r)$, which lies a distance r to its left in the starting body frame. After turning by $\delta\theta$ it is at angle $\delta\theta$ around that circle:

$$t_{\text{arc}} = C + R(\delta\theta)\,(0, -r)^{\top} = \big(r\sin\delta\theta,\ r(1-\cos\delta\theta)\big), \qquad \text{heading } \delta\theta.$$

The step model puts the robot at $(r\,\delta\theta, 0)$ with the same heading. Expanding $\sin\delta\theta = \delta\theta - \delta\theta^3/6$ and $1-\cos\delta\theta = \delta\theta^2/2$:

$$t_{\text{step}} - t_{\text{arc}} = \left(\tfrac{r\,\delta\theta^3}{6},\ -\tfrac{r\,\delta\theta^2}{2}\right) + O(\delta\theta^4).$$

The heading is exact, and the position is wrong *sideways* by $r\,\delta\theta^2/2$. Per step the error is second order, so the model is *first-order accurate*. Over a fixed total turn θ split into $n = \theta/\delta\theta$ steps, the errors add up to $n \cdot O(\delta\theta^2) = O(\theta\,\delta\theta) \to 0$ as $\delta\theta \to 0$.

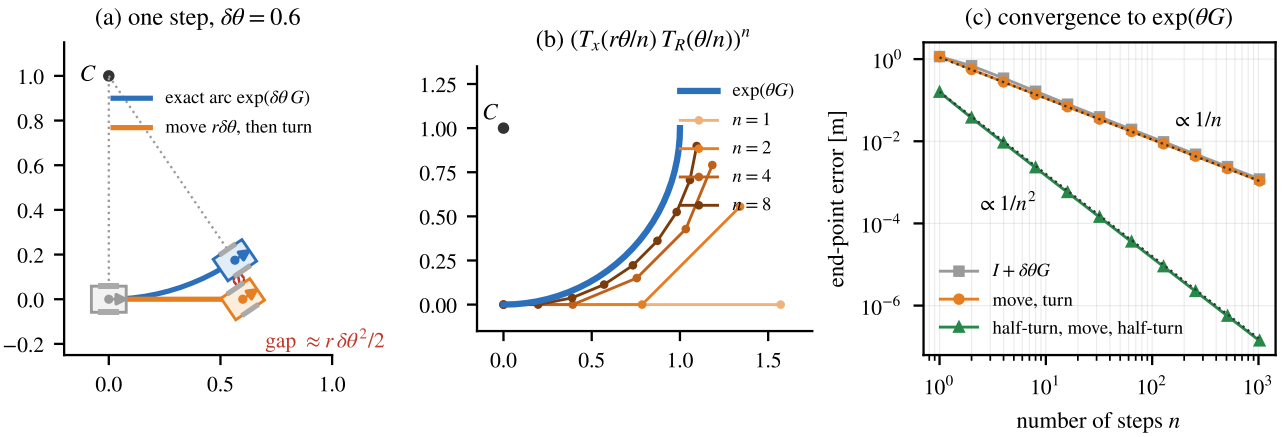


Figure 1.1: (a) One step of "move $r\delta\theta$, then turn" (orange) against the true arc (blue). The heading agrees, but the step model lands $\approx r\delta\theta^2/2$ to the right. (b) Splitting a quarter turn into n such steps: the polygon approaches the arc. (c) End-point error after n steps: first-order steps converge like $1/n$; the symmetric "half-turn, move, half-turn" step converges like $1/n^2$ (§1.5.3).

With $r = 1$ m the sideways gap is 0.124 m for $\delta\theta = 0.5$, 5.0 mm for $\delta\theta = 0.1$ and 0.05 mm for $\delta\theta = 0.01$. Each tenfold reduction of the step shrinks the error a hundredfold, exactly the $r\,\delta\theta^2/2$ predicted.

On the slide "finite steps use the exact ΔT"

The lecture says that finite steps should use the exact ΔT, with $I + \delta\theta G$ as its first-order approximation. Keep in mind that $T_x(\delta s) T_R(\delta\theta)$ is *exact as a matrix* but is itself only a first-order model of an arc: it is the exact pose for a robot that really drives straight and then pivots. For a robot that drives an arc, the exact step is $\exp(\delta\theta\, G)$ (§1.5.5). All three agree to first order, which is all the limit in §1.5.3 needs.

1.5.2 The generator: forward motion and turning in one matrix

For small $\delta\theta$, keep only first-order terms ($\sin\delta\theta \approx \delta\theta$, $\cos\delta\theta \approx 1$):

$$\Delta T \approx I + \delta\theta\, G, \qquad G = \begin{bmatrix} 0 & -1 & r \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \tag{1.2}$$

G does not depend on the step size. It describes the *kind* of motion (a left turn of radius r), while $\delta\theta$ says *how much*. Its two blocks have separate meanings: the upper-left 2×2 block is the rotation generator J, and the last column $(r,0)$ is the forward displacement per radian of turning.

Derivation: G is the derivative of the step, and every reasonable step model has the same one

Define $G = \frac{d}{d\delta\theta}\Delta T(\delta\theta)\big|_{\delta\theta=0}$. For "move, then turn", the product rule at $\delta\theta = 0$ (where both factors are I) gives

$$\frac{d}{d\delta\theta}\left[T_x(r\delta\theta)T_R(\delta\theta)\right]_0 = \underbrace{\begin{bmatrix} 0 & 0 & r \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{T_x'(0)\cdot r} + \underbrace{\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{T_R'(0)} = G.$$

The derivative of a product at the identity is the *sum* of the derivatives. So "turn, then move", "half-turn, move, half-turn", and the exact arc all have the same derivative G. They differ only at second order. This is why the generator, not any particular step model, is the right description of the motion.

J: rotation as "turn the radius into the tangent". Rotating a point p by a small angle moves it perpendicular to its radius, by an amount proportional to its distance from the centre:

$$R(\delta\theta)\,p \approx p + \delta\theta\, Jp, \qquad J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad Jp = (-p_y, p_x).$$

Jp is p turned by $+90°$: it has the same length ($\|Jp\| = \|p\|$) and is orthogonal ($p^\top Jp = -p_xp_y + p_yp_x = 0$). So J maps a radius vector to the tangent of the circle through p. For $p = (1,0)$, $Jp = (0,1)$: a point on the x-axis starts moving straight up. This is the 2-D version of $\omega \times p$ (with $\omega = e_z$), and $J = [e_z]_\times$ restricted to the plane.

G as a velocity field. Apply G to a point in homogeneous coordinates $(p,1)$:

$$G\begin{bmatrix} p \\ 1 \end{bmatrix} = \begin{bmatrix} Jp + (r,0)^\top \\ 0 \end{bmatrix}. \tag{1.3}$$

The result has a 0 in the last slot, so it is a *direction*: the velocity, per radian of turning, of the body point at p. The origin moves forward at $(r,0)$. A point at p additionally swings around by Jp. The point $p = (0,r)$, a distance r to the left, has velocity $J(0,r) + (r,0) = (-r,0) + (r,0) = 0$: it is the centre of the turn. Fig. 1.2 draws this field.

Pitfall: $I + \delta\theta G$ is not a rigid motion

The first-order step has rotation block $\begin{bmatrix} 1 & -\delta\theta \\ \delta\theta & 1 \end{bmatrix}$ with determinant $1 + \delta\theta^2 > 1$. It stretches the plane slightly. Multiplying n of them for a quarter turn gives a rotation block with determinant 3.47 ($n = 1$), 1.17 ($n = 16$), 1.0024 ($n = 1024$). It tends to 1, but at every finite n the result is not in $SE(2)$. Use $I + \delta\theta G$ for analysis and Jacobians. To propagate a pose, use the exponential or at least a product of exact rotations and translations.

1.5.3 From many small steps to the matrix exponential

To turn by a finite angle θ, split it into n identical body-frame steps of $\delta\theta = \theta/n$ and compose them with (1.1):

$$T_n(\theta) = \left(\Delta T_{\theta/n}\right)^n.$$

As $n \to \infty$ the step model stops mattering, and the limit is the exact motion. There are two independent ways to see what that limit is.

Derivation: route 1: the product limit

Every step model from §1.5.2 has the form $\Delta T_{\theta/n} = I + \frac{\theta}{n}G + E_n$ with $\|E_n\| \le c/n^2$. For large n this matrix is close to I, so its matrix logarithm is defined by the convergent series $\log(I+X) = X - \frac{1}{2}X^2 + \dots$, and

$$\log \Delta T_{\theta/n} = \tfrac{\theta}{n}G + O(1/n^2).$$

All n factors are the *same* matrix, so they commute, and $(\Delta T)^n = \exp(n \log \Delta T)$. Therefore

$$T(\theta) = \lim_{n\to\infty} \left(\Delta T_{\theta/n}\right)^n = \lim_{n\to\infty} \exp\left(\theta G + O(1/n)\right) = \exp(\theta G),$$

using continuity of exp. With $\Delta T = I + \frac{\theta}{n}G$ this is the matrix form of the familiar $\lim(1+\frac{x}{n})^n = e^x$. The proof also shows that the $O(1/n^2)$ details of the step model change the answer only by $O(1/n)$, and not at all in the limit.

Derivation: route 2: a motion that repeats itself

"The same body-frame motion again" means: going an amount λ and then a further amount μ, measured from wherever the robot now is, equals going $\lambda+\mu$. In matrices, $T(\lambda+\mu) = T(\lambda)T(\mu)$ with $T(0) = I$, a *one-parameter subgroup*. Differentiate in μ at $\mu = 0$:

$$\frac{dT}{d\lambda}(\lambda) = T(\lambda)\frac{dT}{d\mu}(0) = T(\lambda)G, \qquad T(0) = I.$$

This is a linear ODE with constant coefficients. The series $\exp(\lambda G) = \sum_k \lambda^k G^k/k!$ solves it (differentiate term by term: $\sum_k \lambda^{k-1}G^k/(k-1)! = \exp(\lambda G)\,G$), and linear ODEs have unique solutions, so $T(\lambda) = \exp(\lambda G)$. Since G commutes with $\exp(\lambda G)$, also $\dot{T} = GT$: for a motion that repeats itself, the generator is the same whether it is read in the moving body frame or in the starting frame. For general motions the two differ (body and spatial twists, §2.6.5).

The two routes say the same thing from different sides: the generator is the *velocity* of the motion at its start, and the exponential is what you get by *flowing* along that velocity while it stays constant in the body frame. To predict a pose after the motion, append it on the right:

$$T_{\text{new}} = T_{\text{current}} \exp(\theta G).$$

Worked example: how fast do step models converge?

Quarter turn, $r = 1$ m, so the exact end pose is $\exp(\frac{\pi}{2}G)$: position $(1,1)$, heading 90°. End-point errors after n steps (Fig. 1.1c):

n	$I+\delta\theta G$	move, then turn	half-turn, move, half-turn
1	1.15 m	1.15 m	0.157 m
16	7.99 cm	6.94 cm	0.57 mm
1024	1.21 mm	1.09 mm	0.14 µm

Fitted slopes between $n = 64$ and 1024 are -1.00, -1.00 and -2.00. The first two are first-order methods. The symmetric step $T_R(\frac{\delta\theta}{2})T_x(\delta s)T_R(\frac{\delta\theta}{2})$ is second-order, because its expansion is symmetric under $\delta\theta \to -\delta\theta$ (it is its own reverse). The even-order error terms that a one-sided split leaves behind therefore cancel. This is exactly the "RK2" odometry update used in practice (§1.5.6). The lesson: the exponential is the target every step model is trying to hit, and it is cheap to evaluate exactly (§1.5.5). There is no reason to approximate it.

1.5.4 The exponential of a planar rotation

Before tackling $\exp(\theta G)$ in full, take its rotation block: what is $e^{\theta J}$? Everything follows from one fact: rotating by 90° twice is a half turn,

$$J^2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} = -I.$$

Derivation: $e^{\theta J} = R(\theta)$ from the power series

The powers of J cycle with period four: $J^0 = I$, $J^1 = J$, $J^2 = -I$, $J^3 = -J$, $J^4 = I, \dots$ So the even powers are multiples of I and the odd powers are multiples of J. Group them:

$$\begin{aligned} e^{\theta J} &= I + \theta J + \frac{\theta^2 J^2}{2!} + \frac{\theta^3 J^3}{3!} + \frac{\theta^4 J^4}{4!} + \cdots \\ &= I\Big(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \cdots\Big) + J\Big(\theta - \frac{\theta^3}{3!} + \cdots\Big) = I\cos\theta + J\sin\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} = R(\theta). \end{aligned}$$

Why it had to come out this way. Three equivalent views make the formula unsurprising.

- *Flow.* By route 2 of §1.5.3, $e^{\theta J}p$ solves $\dot{p} = Jp$: the velocity is always the radius turned by 90°, with the same length. Such a point never changes its distance to the origin ($\frac{d}{d\theta}\|p\|^2 = 2p^\top Jp = 0$) and moves at unit angular speed. It moves on a circle, at angle θ after "time" θ.
- *Complex numbers.* Identify (x,y) with $x+iy$. Multiplying by i maps $x+iy \mapsto -y+ix$, which is exactly J. So J *is* i in matrix form, $J^2 = -I$ is $i^2 = -1$, and the derivation above is Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$. Planar rotations are multiplications by unit complex numbers, which also explains why they commute.
- *3-D.* With $\omega = e_z$, the 3×3 skew matrix $[e_z]_\times$ has J as its upper-left block, and Rodrigues' formula (§2.4.4) reduces to the derivation above. The 3-D derivation is the same calculation with the extra axis term $\hat{n}\hat{n}^\top$.

Note what the exponential achieves. The generator θJ is linear in θ and contains no trigonometry; the sines and cosines appear only through the series. A generator is algebraically simpler than the motion it generates. That simplicity is what makes generators the right objects to estimate, interpolate and differentiate.

1.5.5 The exponential of a full twist, and the turn centre

Now the whole generator. Write a general planar generator as $[S] = \begin{bmatrix} \omega J & v \\ 0 & 0 \end{bmatrix}$, with $v = (v_x, v_y)$ the initial velocity of the body origin and ω the turn rate, both per unit of motion amount. The left turn of radius r is $v = (r,0)$, $\omega = 1$, i.e. $[S] = G$. We derive $\exp(\lambda[S])$ twice, once by algebra and once by geometry, because each derivation explains something the other hides.

Derivation: route 1: the series, block by block

Block-multiplying shows that powers keep the same shape, $[S]^k = \begin{bmatrix} (\omega J)^k & (\omega J)^{k-1}v \\ 0 & 0 \end{bmatrix}$ for $k \geq 1$: the bottom row of zeros kills every term that would feed the translation back into itself. Summing the series with $\theta = \lambda\omega$ (the total turn):

$$\exp(\lambda[S]) = \begin{bmatrix} e^{\theta J} & \lambda V(\theta)v \\ 0 & 1 \end{bmatrix}, \qquad V(\theta) = \sum_{k\geq 0} \frac{(\theta J)^k}{(k+1)!}.$$

Split V into even and odd powers exactly as in §1.5.4:

$$V(\theta) = I\Big(1 - \frac{\theta^2}{3!} + \frac{\theta^4}{5!} - \cdots\Big) + J\Big(\frac{\theta}{2!} - \frac{\theta^3}{4!} + \cdots\Big) = \frac{\sin\theta}{\theta} I + \frac{1-\cos\theta}{\theta} J.$$

For the left turn ($v = (r,0)$, $\omega = 1$, $\lambda = \theta$):

$$t(\theta) = \theta V(\theta)\begin{bmatrix} r \\ 0 \end{bmatrix} = \begin{bmatrix} r\sin\theta \\ r(1-\cos\theta) \end{bmatrix}, \qquad T(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & r\sin\theta \\ \sin\theta & \cos\theta & r(1-\cos\theta) \\ 0 & 0 & 1 \end{bmatrix}.$$

For $r = 1$ m and $\theta = \pi/2$: position $(1,1)$ m, heading 90°. This is the exact arc from §1.5.1, now produced by the generator alone. At $\theta \to 0$, use the series ($V \to I$) instead of dividing by θ.

Derivation: route 2: every planar motion with $\omega \neq 0$ is a rotation about one fixed point

By (1.3) the body point at p moves with velocity $\omega Jp + v$. Look for the point that does not move:

$$\omega JC + v = 0 \quad \Longrightarrow \quad C = -\tfrac{1}{\omega}J^{-1}v = \tfrac{1}{\omega}Jv \qquad (J^{-1} = -J).$$

Equivalently $v = -\omega JC = \omega(c_y, -c_x)$. Because the twist is constant in the body frame, and C is fixed in the body frame, C stays fixed for all λ. The motion is therefore a pure rotation about C by $\theta = \lambda\omega$. Writing "rotate about C" as "shift C to the origin, rotate, shift back", with T_C the translation by C:

$$T(\theta) = T_C\, T_R(\theta)\, T_C^{-1} = \begin{bmatrix} R(\theta) & C - R(\theta)C \\ 0 & 1 \end{bmatrix}, \qquad t(\theta) = (I - R(\theta))C.$$

For the left turn $C = J(r,0) = (0,r)$, and $t(\pi/2) = (0,r) - (-r,0) = (r,r)$, the same answer. The two routes agree in general because $\theta V(\theta) = \sin\theta\, I + (1-\cos\theta)J = (I - R(\theta))J$, which is checked by expanding $R(\theta)J = \cos\theta J - \sin\theta I$.

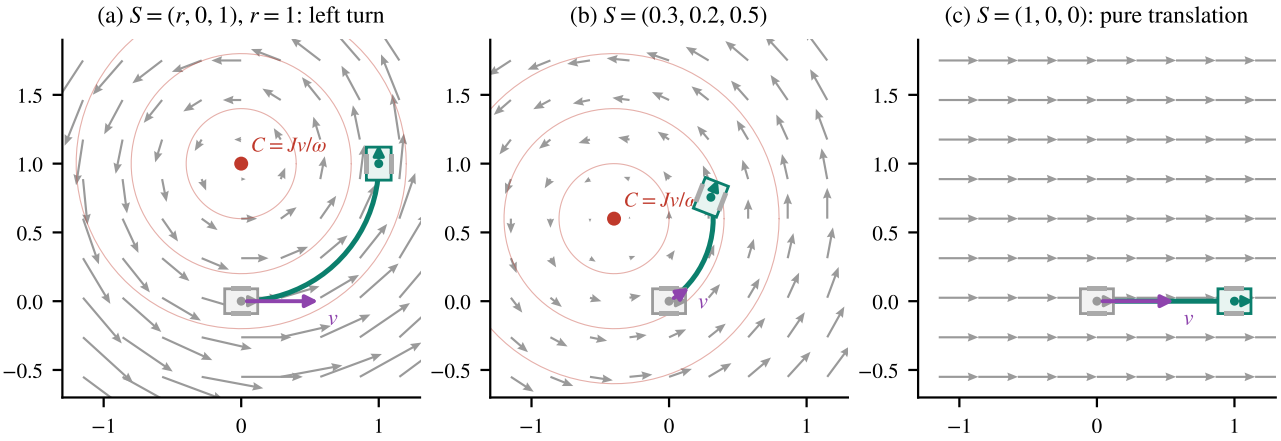


Figure 1.2: The velocity field $\omega Jp + v$ of three twists, with the motion $\exp(\lambda[S])$ of the robot. (a) Left turn: every body point circles $C = (0,1)$. (b) A twist with sideways velocity (an omnidirectional base): the centre moves to $C = (-0.4, 0.6)$, still at Jv/ω. (c) $\omega = 0$: the centre is at infinity, and every point moves the same way.

What this says. In the plane, *every* rigid motion is either a rotation about a single point (when $\omega \neq 0$) or a pure translation (when $\omega = 0$, the limit $C \to \infty$). This is the planar case of Chasles' theorem (§2.6.4); in the plane there is no room for the extra slide along the axis. It also explains the slide's warning that $t(\theta)$ need not equal θv. The vector v is only the *initial* direction of the origin, and the origin then curves around C. After a half turn ($r = 1$, $\theta = \pi$) the robot is at $t = (0,2)$, while $\theta v = (\pi, 0)$ points the wrong way entirely. $V(\theta)$ is the matrix that converts the "straight-line" displacement λv into the true chord. Using the half-angle identities $\sin\theta = 2\sin\frac{\theta}{2}\cos\frac{\theta}{2}$ and $1 - \cos\theta = 2\sin^2\frac{\theta}{2}$, it factors as

$$V(\theta) = \frac{2\sin(\theta/2)}{\theta}\, R(\theta/2). \tag{1.4}$$

The scalar is (chord length)/(arc length) for an arc of angle θ, and $R(\theta/2)$ turns the initial direction to the chord's direction, which is half the total turn. For the quarter turn, $V = 0.900\,R(45^\circ)$: the arc of length $\pi/2$ becomes a chord of length $0.900 \cdot \pi/2 = \sqrt{2}$ pointing at 45°.

Worked example: where is the instantaneous centre of rotation?

A differential-drive robot commands $v = 0.4$ m/s, $\omega = 0.5$ rad/s, i.e. body twist $(0.4, 0, 0.5)$ per second. Its turn centre is $C = Jv/\omega = (0, 0.8)$ m: on the wheel axle line, 0.8 m to the left. A mecanum or omni-wheel base can also command v_y. The twist $(0.3, 0.2, 0.5)$ has $C = (-0.4, 0.6)$ m, radius 0.721 m, behind and to the left. Both satisfy $v + \omega JC = 0$ exactly.

Pitfall: the centre must lie where the wheels allow it

A differential drive cannot move sideways, so its twist always has $v_y = 0$ and C lies on the axle line. A car-like (Ackermann) vehicle can only turn about points on the extension of its rear axle. A planner that outputs twists with $v_y \neq 0$ for such a robot is asking for an instantaneous centre the wheels cannot produce: the robot will skid or the controller will saturate. Checking where $C = Jv/\omega$ lies is a one-line sanity test for any commanded twist.

1.5.6 Twists: naming, scaling and units

The generator deserves a name and a coordinate vector. A planar *twist* is

$$S = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}, \qquad [S] = \begin{bmatrix} 0 & -\omega & v_x \\ \omega & 0 & v_y \\ 0 & 0 & 0 \end{bmatrix} \in \mathfrak{se}(2),$$

linear part first (the book's convention for 3-D twists as well, chapter 2). The set $\mathfrak{se}(2)$ of all such matrices is a 3-dimensional vector space, the *Lie algebra* of $SE(2)$. It is the space of velocities of planar rigid motions: by route 2 of §1.5.3, for any motion $T(\lambda)$ with $T(0) = I$, $\frac{dT}{d\lambda}\big|_0$ has exactly this shape. The components are expressed in the *starting body frame*.

Unit twists and the motion amount. A twist mixes units (metres and radians), so "unit length" cannot mean $\|S\| = 1$. It means instead that λ gets a clean physical meaning:

- *Unit rotational twist*: $\omega = 1$. Then $\lambda = \theta$ is the turn angle in radians and v is the displacement per radian (for the left turn, $\hat{S} = (r, 0, 1)$, and $|v| = r$ is the turn radius).
- *Unit translational twist*: $\omega = 0$, $\|v\| = 1$. Then $\lambda = d$ is the distance in metres, and $\exp(d[\hat{S}_p])$ with $\hat{S}_p = (1, 0, 0)$ is just $T_x(d)$.

Any twist factors as $S = \lambda\hat{S}$ (amount times kind), and $T(\lambda) = \exp([S]) = \exp(\lambda[\hat{S}])$ with $T(0) = I$ and $\frac{dT}{d\lambda}\big|_0 = [\hat{S}]$. Here λ is a motion *amount*, not a time. When the robot moves with a constant body velocity twist $\mathcal{V} = (v_x, v_y, \omega)$ in m/s and rad/s for a time Δt, the same machinery applies with $S = \mathcal{V}\Delta t$.

Pitfall: scaling the wrong thing

A left turn of radius r by angle θ is $S = \theta\,(r, 0, 1) = (r\theta, 0, \theta)$, not $(r, 0, \theta)$. The second has translation part $v = (r, 0)$ "spent" over the whole turn, i.e. a radius of r/θ. For $r = 1$, $\theta = \pi/2$ it lands at $(0.637, 0.637)$ instead of $(1, 1)$, with the correct heading, so a heading-only check will not catch it. This exact bug happened while preparing this section's examples. Always scale the *whole* unit twist.

Worked example: differential-drive odometry: exact, Euler and RK2

Kinematics. A differential-drive robot has wheels at $y = \pm b/2$ on its axle (track width b). Over one encoder interval the wheels roll arc lengths Δs_L, Δs_R. The body origin travels the average and the heading changes by the difference over the lever arm:

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \qquad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{b}, \qquad S_k = (\Delta s,\, 0,\, \Delta\theta).$$

If the wheel speeds are constant during the interval (the usual zero-order-hold assumption), the body twist is constant, and by §1.5.3 the *exact* update is $T_{k+1} = T_k \exp([S_k])$. Its turn centre is $C = J(\Delta s, 0)^\top / \Delta\theta = (0, \Delta s/\Delta\theta)$, on the axle line, at radius $\frac{b}{2}\frac{\Delta s_R + \Delta s_L}{\Delta s_R - \Delta s_L}$.

Experiment. A robot with $b = 0.30$ m drives a slalom at 0.6 m/s with turn rate $0.8\cos(2\pi t/8\,\text{s})$ rad/s for 60 s (36 m). Three integrators receive the same encoder increments: the exponential; "Euler" ($x \mathrel{+}= \Delta s\cos\theta$, $y \mathrel{+}= \Delta s\sin\theta$, then $\theta \mathrel{+}= \Delta\theta$, i.e. move, then turn); and "RK2" (the same with heading $\theta + \Delta\theta/2$, i.e. half-turn, move, half-turn). Ground truth is the continuous motion integrated by an adaptive ODE solver. Maximum position error over the run:

encoder rate	exponential	RK2	Euler
50 Hz	exact	0.017 cm	0.59 cm
10 Hz	exact	0.42 cm	2.9 cm
2 Hz	exact	10.5 cm	21 cm

The exponential is exact at every rate, because it *is* the solution of the ODE when the twist is constant: at 10 Hz it agrees with the high-accuracy ODE solution to 2×10^{-7} m, which is the solver's own accuracy. Euler's error grows linearly with the step, and RK2's quadratically, as predicted in §1.5.3.

Lesson. At 50–100 Hz any of the three is far below wheel slip and calibration errors. But integration error is *systematic*: it does not average out. It matters whenever increments are large: low-rate odometry, keyframe-to-keyframe motion in a pose graph (§1.4), or replaying a long log at a coarse rate. ROS 2's `diff_drive_controller` uses the same exact arc (its `integrateExact` computes $r = \Delta s/\Delta\theta$ and moves along the circle). It switches to the RK2 update when $|\Delta\theta| < 10^{-6}$, because $r = \Delta s/\Delta\theta$ blows up as the arc straightens. The closed form $\exp([S])$ with the series for $V(\theta)$ near $\theta = 0$ needs no such branch.

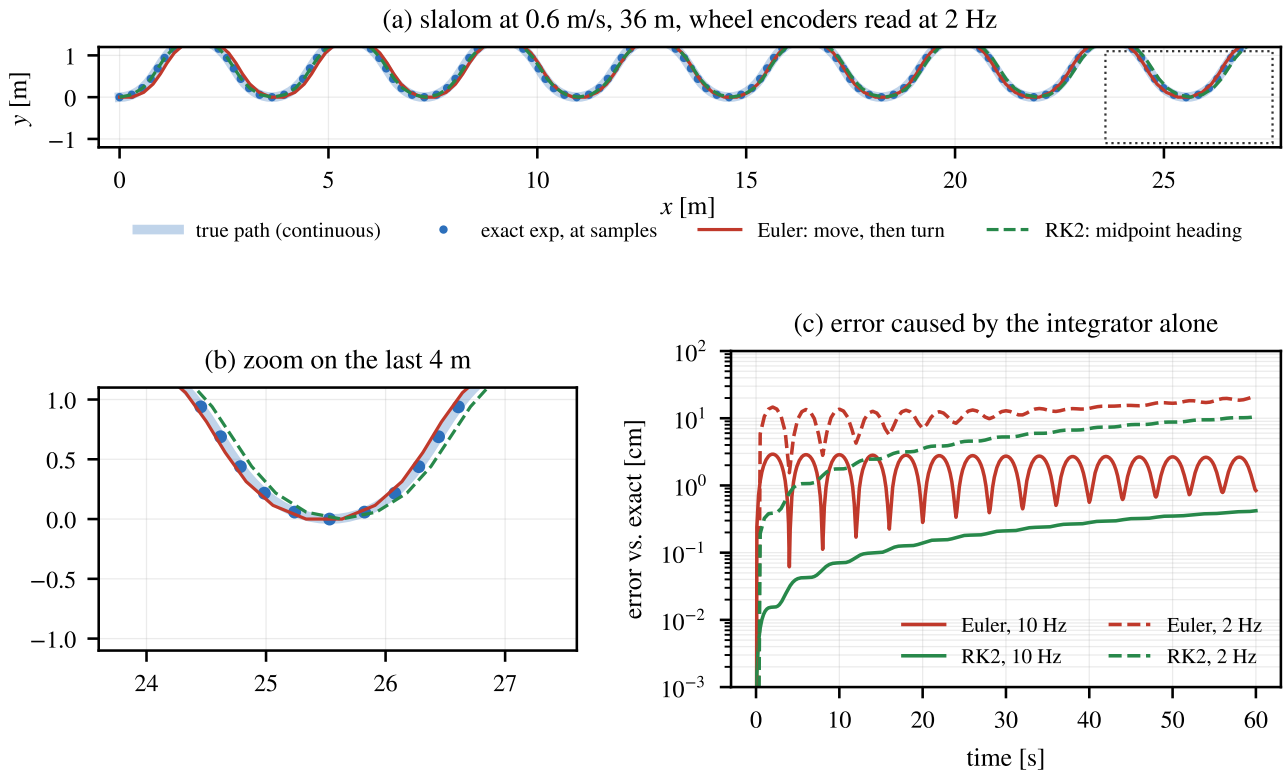


Figure 1.3: Odometry integration with identical encoder data. (a,b) At 2 Hz, Euler (red) cuts the inside of every bend and drifts. RK2 (green dashed) is close. The exponential lands exactly on the true path at every sample. (c) Error relative to the exact solution: integrator error alone, with no slip or noise.

1.5.7 Logarithms: from a pose back to a twist

The exponential turns a twist into a pose. Estimation needs the reverse: given two poses, which twist carries one into the other? That is the *logarithm*. It exists, but it cannot be unique, because the exponential is not one-to-one.

Rotations. For a rotation $R \in SO(2)$ we want $L \in \mathfrak{so}(2) = \{\theta J\}$ with $\exp(L) = R$. Since $e^{\theta J} = R(\theta)$, the answer is $L = \theta J$, where θ is *any* angle with $\cos\theta = r_{11}$, $\sin\theta = r_{21}$:

$$L = \log R = \begin{bmatrix} 0 & -\theta \\ \theta & 0 \end{bmatrix}, \qquad L^\top = -L, \qquad \operatorname{vex}(L) = \theta.$$

Angles differing by 2π give the same rotation, $R(0) = R(2\pi) = I$: a robot that has spun once on the spot is oriented exactly as before, and the matrix keeps no record of the turn. A logarithm must therefore pick a *branch*. The principal branch is $\theta = \operatorname{atan2}(r_{21}, r_{11}) \in (-\pi, \pi]$, the smallest turn that does the job.

Derivation: the logarithm of a pose, in closed form

We want $S = (\rho, \theta)$ (with $\rho = \lambda v$ the translational part) such that $\exp([S]) = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$. By §1.5.5, $\exp([S]) = \begin{bmatrix} R(\theta) & V(\theta)\rho \\ 0 & 1 \end{bmatrix}$. So:

1. the rotation fixes $\theta = \operatorname{atan2}(r_{21}, r_{11})$ (choosing the branch here chooses it for everything);
2. the translation fixes $\rho = V(\theta)^{-1} t$.

Inverting V is easy using its factored form (1.4), $V = \frac{2\sin(\theta/2)}{\theta} R(\theta/2)$, because the inverse of a rotation is its transpose:

$$V(\theta)^{-1} = \frac{\theta}{2\sin(\theta/2)} R(-\theta/2) = \frac{\theta}{2} \cot\frac{\theta}{2}\, I - \frac{\theta}{2} J.$$

It exists whenever $\sin(\theta/2) \neq 0$, i.e. for all θ in the principal branch (use $V^{-1} \to I - \frac{\theta}{2}J$ as $\theta \to 0$). Geometrically: undo the half-turn to get the initial direction, then stretch the chord back to the arc length. The result is $S = \operatorname{vexa}(\log T) = (\rho_x, \rho_y, \theta)$, where vexa reads off the three independent entries of a matrix in $\mathfrak{se}(2)$.

Worked example: the logarithm does not recover the motion history

A robot drives three quarters of a left circle of radius 1 m: $S = \frac{3\pi}{2}(1,0,1)$. It ends at $t = (-1,1)$ facing $-90°$. The principal logarithm of that pose is

$$\operatorname{vexa}(\log T) = \left(-\tfrac{\pi}{2},\ 0,\ -\tfrac{\pi}{2}\right) = \tfrac{\pi}{2}(-1,0,-1)\,:$$

a quarter turn to the *right*, driving *backwards*, about the same centre $C = (0,1)$ (Fig. 1.4). Both twists produce the same pose. The logarithm returns the shortest one (90° instead of 270°; arc length $\pi/2$ instead of $3\pi/2$). `scipy.linalg.logm` returns the same principal branch. A full circle is worse still: $\exp(2\pi[(1,0,1)]) = I$, and its logarithm is 0. The robot drove 2π m and the logarithm reports that it never moved.

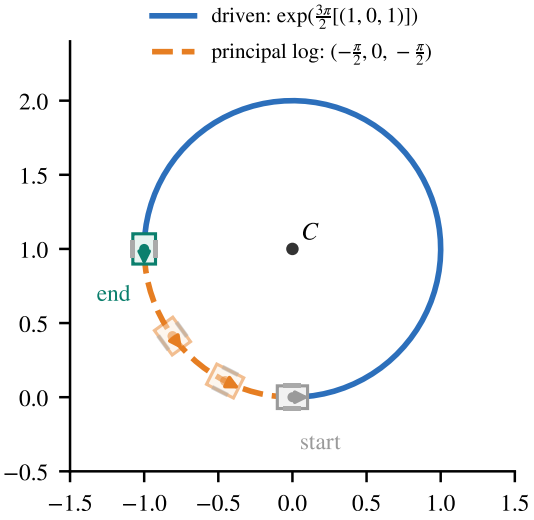


Figure 1.4: Two twists, one pose. The robot drove 270° forward around C (solid). The principal logarithm of the end pose describes a 90° *reverse* arc about the same centre (dashed, intermediate poses faded). Both are valid generators. Only the motion history (time stamps, intermediate poses, or the knowledge that steps are small) can tell them apart.

Worked example: a pose-graph residual

In a pose graph (§1.4), an edge stores a measured relative pose Z_{ab} between nodes with estimated poses T_a, T_b. The estimate predicts $T_a^{-1}T_b$. The mismatch is the pose $Z_{ab}^{-1}T_a^{-1}T_b$, which is I when estimate and measurement agree. The logarithm turns it into a 3-vector the optimiser can square:

$$e_{ab} = \operatorname{vexa}\left(\log(Z_{ab}^{-1}T_a^{-1}T_b)\right) \in \mathbb{R}^3.$$

With $T_a = (2,1,30°)$, $T_b = (2.9,1.6,41°)$ (position in m, heading) and $Z_{ab} = (1.05,0.05,10°)$: the predicted relative pose is $(1.079,0.070,11°)$, and $e_{ab} = (0.0325\,\text{m},\ 0.0139\,\text{m},\ 0.01745\,\text{rad} = 1.00°)$. The heading error is just the difference of headings, as it must be in the plane. The translational part is *not* the naive difference $(0.029,0.020)$: it is expressed in the measurement's frame and corrected by V^{-1}. For small errors these effects are second order, and e_{ab} behaves like an ordinary vector, which is exactly what least squares needs.

Pitfall: branch jumps

The principal logarithm is discontinuous at $\theta = \pm\pi$: an error that drifts from 179° to 181° jumps to −179°. In a residual this is harmless only if errors stay well inside $(-\pi,\pi)$, which they do near convergence but may not with a bad initial guess or a wrong loop closure. Also, `scipy.linalg.logm` is a general-purpose algorithm: it can return a complex array with tiny imaginary parts and is slow. For $SE(2)$, use the closed form above with atan2.

1.5.8 Twists do not commute: the Lie bracket and parallel parking

Twists form a vector space, so they can be added. It is tempting to conclude that $\exp([S_1])\exp([S_2]) = \exp([S_1+S_2])$, as for numbers. It is false, and the step model of §1.5.1 already showed it: "move, then turn" $= \exp(\delta s[A])\exp(\delta\theta[B])$, with $A = (1,0,0)$ (forward) and $B = (0,0,1)$ (pivot), lands $r\,\delta\theta^2/2$ to the side of the arc $\exp(\delta s[A] + \delta\theta[B])$. The failure has a precise measure.

Derivation: the commutator of two small motions

Drive the four-move loop "forward ε, turn ε, back ε, turn back ε". Expand each factor to second order, $e^{\varepsilon X} = I + \varepsilon X + \frac{\varepsilon^2}{2}X^2 + O(\varepsilon^3)$, and multiply:

$$e^{\varepsilon A}e^{\varepsilon B}e^{-\varepsilon A}e^{-\varepsilon B} = I + \varepsilon^2\big(AB - BA\big) + O(\varepsilon^3) = I + \varepsilon^2[A,B] + O(\varepsilon^3).$$

The first-order terms cancel ($+A+B-A-B$). At second order, the four squares contribute $A^2 + B^2$, and the six ordered cross terms contribute $AB - A^2 - AB - BA - B^2 + AB = AB - BA - A^2 - B^2$. The squares cancel, and what survives is the *Lie bracket* $[A,B] = AB - BA$. For any two planar twists, block multiplication gives

$$\big[[S_1],[S_2]\big] = \begin{bmatrix} \omega_1\omega_2 J^2 - \omega_2\omega_1 J^2 & \omega_1 J v_2 - \omega_2 J v_1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \omega_1 J v_2 - \omega_2 J v_1 \\ 0 & 0 \end{bmatrix}.$$

The bracket is always a *pure translation* (planar rotations commute, so the rotation parts cancel). For forward and pivot, $\omega_A = 0$, $v_A = (1,0)$, $\omega_B = 1$, $v_B = 0$: $[A,B] \to -J(1,0) = (0,-1,0)$, i.e. *straight sideways*.

This is remarkable: a differential-drive robot can never move sideways, yet a loop of forward and turning moves produces exactly the forbidden direction, at second order in the move size. The same expansion gives the Baker–Campbell–Hausdorff series, $\log(e^X e^Y) = X + Y + \frac{1}{2}[X,Y] + \ldots$. With $X = \delta s[A]$, $Y = \delta\theta[B]$ and $\delta s = r\delta\theta$, the bracket term $-\frac{1}{2}r\delta\theta^2$ sideways is precisely the gap between "move, then turn" and the true arc found in §1.5.1.

Worked example: parallel parking a differential-drive robot

A robot must shift sideways into a docking slot without any lateral motion. One cycle with $\varepsilon = 0.2$ (0.2 m and 0.2 rad $\approx 11.5°$) moves it by $(0.0040, -0.0397)$ m with heading restored exactly. The bracket predicts $\varepsilon^2[A,B] = (0,-0.04)$; the small forward drift is the $O(\varepsilon^3)$ term. Ten cycles give $(0.040, -0.397)$ m: about 40 cm sideways after driving 4 m back and forth (Fig. 1.5). For $\varepsilon = 0.1$ the net shift is $(0.0005, -0.0100)$: halving ε quarters the sideways gain, as ε^2 predicts. To shift left, reverse the order of the turns. A car parallel-parks by the same principle, with steering instead of pivoting.

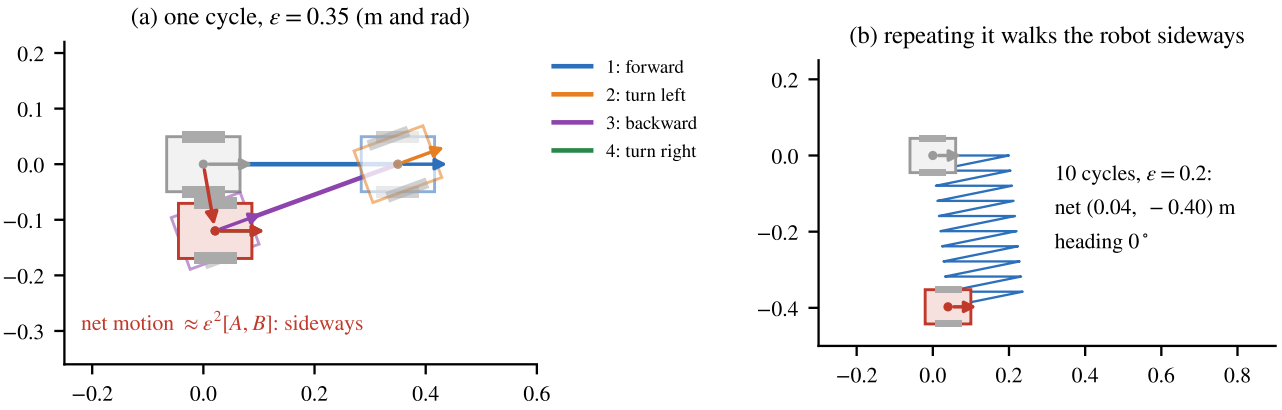


Figure 1.5: (a) Forward, turn left, backward, turn right: the four moves do not cancel, and the robot ends up shifted to its right by $\approx \varepsilon^2$ with its original heading. (b) Repeating the cycle walks the robot sideways. This is the Lie bracket $[A,B]$ made physical.

Why this matters beyond parking

The bracket decides *reachability*. The velocity twists a differential drive can execute span only $(v_x, 0, \omega)$: two of three directions. Their brackets supply the third. Chow's theorem then guarantees that the robot can reach every pose, even though it cannot move in every direction (it is nonholonomic but controllable). Two rotations about different centres $C_1 \neq C_2$ also bracket to a pure translation: for unit rotations about $(0,0)$ and $(1,0)$ the bracket is $(1,0,0)$. In 3-D the same bracket appears as $[a]_\times[b]_\times - [b]_\times[a]_\times = [a \times b]_\times$ (§2.4.6). It is the reason rotation vectors add only to first order.

Where this goes next. Every idea in this section carries over to 3-D with the same proofs: J becomes $[\omega]_\times$, $V(\theta)$ gains an axis term, the turn centre becomes Chasles' screw axis, and the logarithm still picks the shortest generator. Chapter 2 develops this in §§2.4.5–2.4.6 (rotations) and §§2.6.4–2.6.6 (twists).

Chapter 2

Orientation, Rotations and Rigid-Body Motion

Key idea: the one sentence behind the whole chapter

The orientation of a rigid body is *one* physical fact: how its attached frame sits relative to another frame. Rotation matrices, Euler angles, axis–angle, rotation vectors and quaternions are five different *coordinate systems on the same curved space* $SO(3)$. Each one makes some operations easy and some impossible. Adding position turns $SO(3)$ into $SE(3)$, and the same story repeats one level up with homogeneous matrices and twists. Almost every bug with rotations in real robot code comes from forgetting that this space is curved (you cannot add rotations like vectors) or from mixing up which frame a number is expressed in.

Roadmap. §2.1 sets up the language (frames, active vs. passive, why 3-D rotations do not commute). §2.2 builds rotation matrices from nothing but "where do the axes go" and derives $SO(3)$. §2.3 parameterises $SO(3)$ by three angles and shows exactly why that must fail somewhere (gimbal lock). §2.4 finds the parameterisation that comes from physics, constant-rate spinning, which gives Rodrigues' formula, the exponential map and the Lie algebra $\mathfrak{so}(3)$; this is the toolset of modern estimation. §2.5 shows quaternions as the cleanest computational encoding of the same idea and explains the famous half-angle. §2.6 adds translation: homogeneous transforms, $SE(3)$, screws, twists, the adjoint and product-of-exponentials kinematics. §2.7 compares everything, gives a conversion map, a formula sheet, and practice problems with answers.

Every numerical example in this chapter was computed and checked in code (numpy/scipy), so the numbers can be used to test your own implementations.

2.1 Prerequisites: the language of 3-D rotation

2.1.1 Vectors are arrows; coordinates are a choice

A vector v (a displacement, a velocity, a gravity direction) exists independently of any frame. Coordinates appear only once we pick an orthonormal, right-handed basis $\{a_x, a_y, a_z\}$:

$$v = v_1 a_x + v_2 a_y + v_3 a_z, \qquad v_i = a_i \cdot v \quad (\text{because } a_i \cdot a_j = \delta_{ij}).$$

So a coordinate is a *projection* onto a basis direction. Hold onto this: every entry of a rotation matrix will turn out to be one of these projections. The same arrow has different coordinate triples ${}^{A}v$ and ${}^{B}v$ in different frames; most of this chapter is about converting between them.

A *point* is different from a vector: it has a location, so its coordinates depend on the frame's origin as well as its axes. Rotations act identically on both; translations move points but leave vectors (directions) untouched. This distinction becomes the "1 vs. 0" in homogeneous coordinates (§2.6.2).

2.1.2 Dot product, cross product, and the right-hand rule

Dot product $a \cdot b = \|a\|\|b\| \cos\angle(a,b)$: defined purely by lengths and angles, so any map that preserves lengths and angles preserves dot products. **Cross product** $a \times b$: the vector perpendicular to both, of length $\|a\|\|b\| \sin\angle(a,b)$, direction given by the right-hand rule. Two facts are used constantly:

$$a \times (b \times c) = b\,(a \cdot c) - c\,(a \cdot b) \qquad (\text{"BAC–CAB"}), \tag{2.1}$$

$$R(a \times b) = (Ra) \times (Rb) \quad \text{for every proper rotation } R. \tag{2.2}$$

Eq. (2.2) holds because the cross product is built from lengths, angles and handedness, and a proper rotation preserves all three. (A mirror reflection preserves lengths and angles but flips handedness, and (2.2) then fails by a sign.) It has a useful matrix form. Since $a \times b$ is linear in b, there is a matrix $[a]_\times$ with $[a]_\times b = a \times b$ (written out in §2.4.3). Then (2.2) gives

$$[Ra]_\times b = (Ra) \times b = R\big(a \times R^\top b\big) = R[a]_\times R^\top b \quad \Longrightarrow \quad \boxed{[Ra]_\times = R[a]_\times R^\top}. \tag{2.3}$$

In words: rotating the axis of a cross product is the same as conjugating its matrix. This identity will reappear in Euler-rate derivations, angular velocity, and the adjoint map.

2.1.3 Orthogonal matrices: maps that keep geometry

A square matrix Q is *orthogonal* if $Q^\top Q = I$. Three equivalent readings:

(i) its columns are unit length and mutually perpendicular ($Q^\top Q = I$ *is* the table of column dot products);
(ii) it preserves dot products: $(Qx)\cdot(Qy) = x^\top Q^\top Qy = x\cdot y$, hence lengths, distances and angles;
(iii) its inverse is free: $Q^{-1} = Q^\top$.

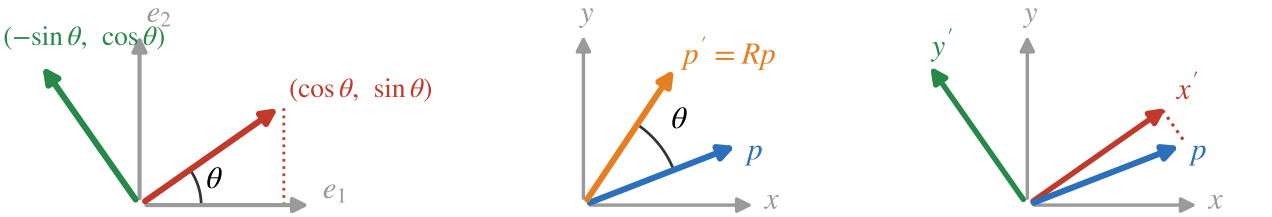
Taking determinants of $Q^\top Q = I$ gives $(\det Q)^2 = 1$, so $\det Q = \pm 1$. $+1$: a proper rotation (handedness kept). -1: a rotation combined with a reflection (a left hand becomes a right hand). Physical motions of a rigid body are always $+1$.

2.1.4 Active vs. passive: one matrix, two readings

(a) where the basis vectors go
$\Rightarrow$ columns of $R_z(\theta)$

(b) active: the vector moves, the frame stays

(c) passive: the vector stays, the frame moves



coords of p in new frame: $R^\top p$

Figure 2.1: (a) A rotation is fully determined by where it sends the basis vectors; those images are its columns. (b) Active reading: the vector moves, the frame is fixed, $p' = Rp$. (c) Passive reading: the vector is fixed, the frame turns by θ; the same physical p now has coordinates $R^\top p$ in the new frame.

- **Active (alibi):** the object moves. $p' = Rp$ are the new coordinates of the moved vector, in the same frame.
- **Passive (alias):** nothing moves; we re-describe the same vector in a rotated frame. If the frame is rotated by R, the coordinates change by $R^\top$.

These are not two different physics; they are inverse readings of the same matrix. In these notes, R_{AB} is read passively as "convert B-coordinates to A-coordinates", which is simultaneously the active rotation that carries A's axes onto B's axes. If you ever get a sign or transpose wrong, it is almost always because the code author used the other reading.

2.1.5 World frame vs. body frame

The *world* (spatial, fixed, inertial) frame is attached to the environment. The *body* frame is glued to the robot and moves with it. Sensors report in body frames (an IMU measures angular rate and specific force in its own axes; a camera measures pixels in its own optical frame), while maps, goals and gravity live in the world frame. Converting between the two, at every time step, is the main job of rotations in practice.

2.1.6 Why 3-D rotations do not commute

In 2-D every rotation is about the same axis (the one sticking out of the plane), so $R(\alpha)R(\beta) = R(\alpha+\beta) = R(\beta)R(\alpha)$. In 3-D, rotations about *different* axes interfere: the first rotation moves the axis that the second one would have used. Fig. 2.2 shows the same two 90° turns applied in opposite orders ending in completely different orientations. Consequences: (a) the order in a product of rotations is meaningful, (b) "sum of angles" is meaningless unless the axes agree, (c) the set of rotations cannot be a vector space. Everything in §2.4 is about dealing with (c) correctly.

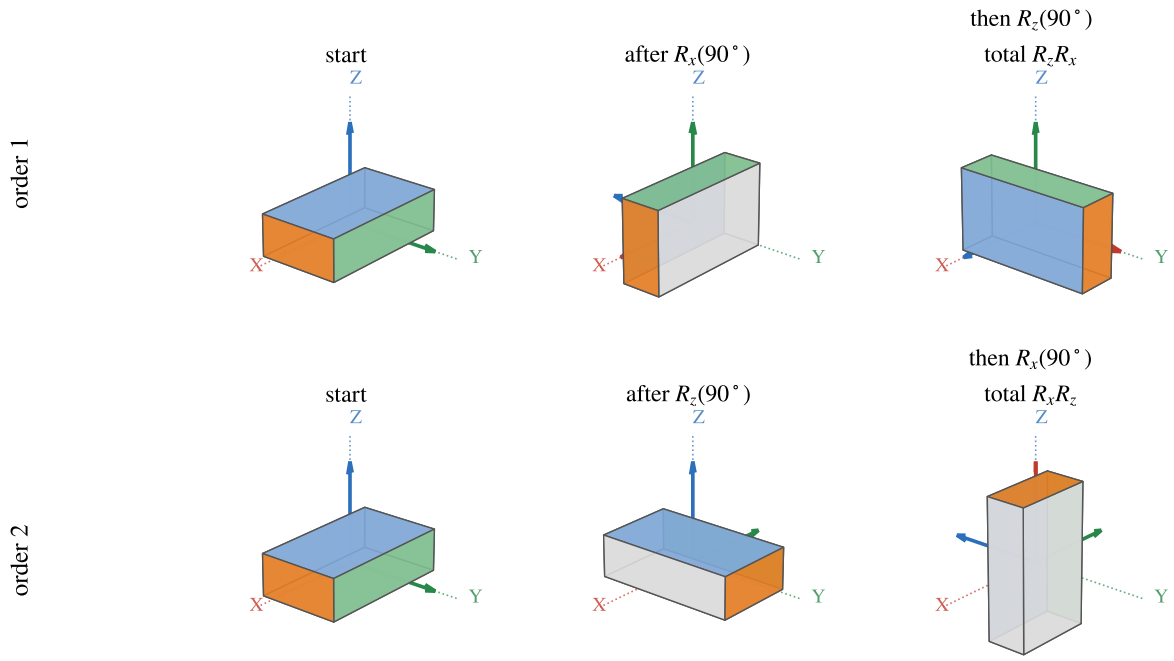


Figure 2.2: The same two 90° rotations about the fixed X and Z axes, in opposite orders. Orange face $= +x$ face of the box, blue $= +z$, green $= +y$. Final orientations differ: $R_zR_x \neq R_xR_z$. (In §2.5 we compute both products: they are 120° rotations about $(1,1,1)/\sqrt{3}$ and $(1,-1,1)/\sqrt{3}$.)

2.1.7 Vocabulary

Term	What it is	Mathematical object
Orientation (attitude)	the *state*: how a body frame is turned relative to a reference	$R \in SO(3)$ (3 DOF)
Rotation	the *operation* that changes orientation (or converts coordinates)	also $R \in SO(3)$
Position	where the body origin is	$t \in \mathbb{R}^3$ (3 DOF)
Pose (configuration)	position + orientation	$T \in SE(3)$ (6 DOF)
Displacement	the relative motion between two poses	$T_1^{-1}T_2 \in SE(3)$
Angular velocity / twist	instantaneous rate of rotation / of rigid motion	$\boldsymbol{\omega} \in \mathbb{R}^3$ / $\xi \in \mathbb{R}^6$

The fact that "state" and "operation" are the same kind of object is special: an orientation *is* the rotation that takes the reference frame to the body frame. That is why a single group, $SO(3)$, serves both roles.

2.2 Orientation in 3-D

2.2.1 Orientation is a relation between two frames

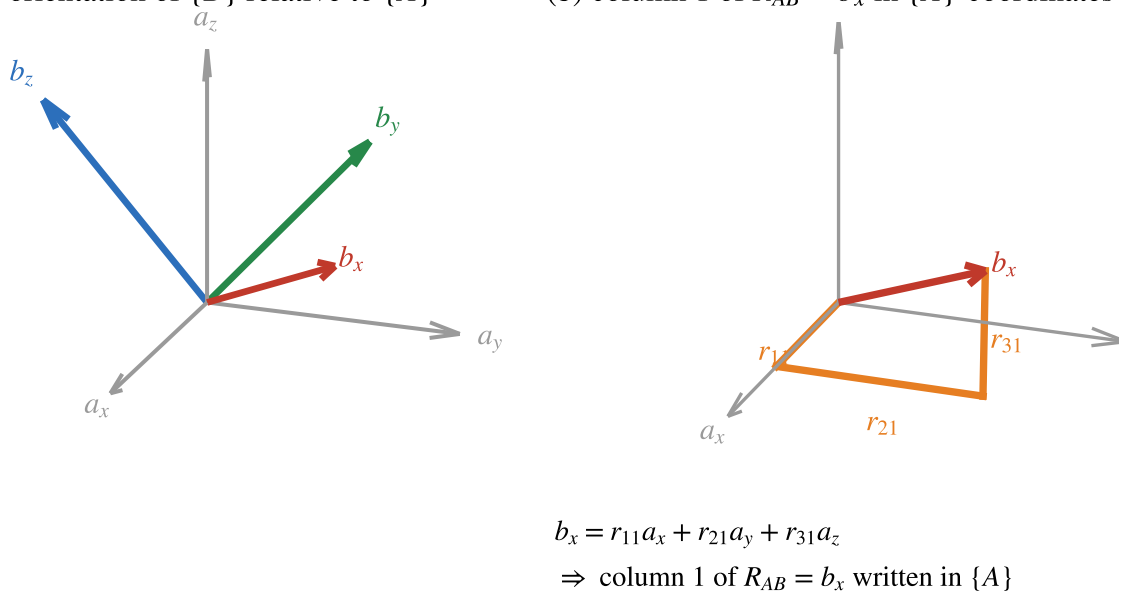
A single direction vector (say, where the nose points) is *not* an orientation: you can still spin the body about that direction. To pin the body down you need its whole frame $\{b_x, b_y, b_z\}$, and that frame only means something relative to another frame $\{a_x, a_y, a_z\}$. Orientation is therefore a *relation between two frames*, and the natural object to store is: "the body axes, written in reference coordinates".

2.2.2 The rotation matrix from first principles

Write each body axis in A-coordinates and stack them as columns:

$$R_{AB} = \begin{bmatrix} {}^{A}b_x & {}^{A}b_y & {}^{A}b_z \end{bmatrix} = \begin{bmatrix} a_x\cdot b_x & a_x\cdot b_y & a_x\cdot b_z \\ a_y\cdot b_x & a_y\cdot b_y & a_y\cdot b_z \\ a_z\cdot b_x & a_z\cdot b_y & a_z\cdot b_z \end{bmatrix}, \qquad r_{ij} = a_i\cdot b_j = \cos\angle(a_i, b_j). \tag{2.4}$$

(a) orientation of $\{B\}$ relative to $\{A\}$ (b) column 1 of $R_{AB} = {}^{A}b_x$ in $\{A\}$-coordinates



$$b_x = r_{11}a_x + r_{21}a_y + r_{31}a_z$$
$$\Rightarrow \text{ column 1 of } R_{AB} = b_x \text{ written in } \{A\}$$

Figure 2.3: (a) Reference frame $\{A\}$ (grey) and body frame $\{B\}$ (colour) sharing an origin. (b) The first column of R_{AB} is the arrow b_x decomposed along a_x, a_y, a_z; each component is a projection $r_{i1} = a_i \cdot b_x$.

This is why rotation matrices are also called *direction-cosine matrices*. Why does it convert coordinates? Take any vector with body coordinates ${}^{B}p = (p_1, p_2, p_3)$, meaning $p = p_1 b_x + p_2 b_y + p_3 b_z$. Writing each b_j in A-coordinates and summing:

$$ {}^{A}p = p_1\,{}^{A}b_x + p_2\,{}^{A}b_y + p_3\,{}^{A}b_z = R_{AB}\,{}^{B}p. $$

That is literally the definition of matrix–vector multiplication (a weighted sum of columns). Nothing else is needed.

Key idea

Column j of R_{AB} is axis j of $\{B\}$ seen from $\{A\}$; row i of R_{AB} is axis i of $\{A\}$ seen from $\{B\}$. If you can say which frame's axes are in the columns, you will never confuse R_{AB} with R_{BA}.

2.2.3 Elementary rotations

Derivation: $R_z(\theta), R_x(\theta), R_y(\theta)$

Rotate by θ about z (counter-clockwise looking down $+z$). From Fig. 2.1(a), $e_1 \mapsto (\cos\theta, \sin\theta, 0)$, $e_2 \mapsto (-\sin\theta, \cos\theta, 0)$, and e_3 is on the axis so it stays. Those images are the columns:

$$R_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}.$$

R_x follows from R_z by the cyclic relabelling $x \to y \to z \to x$ (which preserves handedness). For R_y the cyclic pair is (z, x), not (x, z), because $z \times x = +y$. Writing the block in the order (x, z) therefore moves the minus sign to the lower-left. This is not a typo to memorise; it is the right-hand rule.

2.2.4 Why $R^{-1} = R^{\top}$, why $\det R = +1$: the group $SO(3)$

Derivation: the defining properties of a rotation

(1) Orthogonality. A physical rotation preserves lengths and angles, hence dot products: $(Rx) \cdot (Ry) = x \cdot y$ for all x, y. Then $x^{\top}(R^{\top}R)y = x^{\top}y$ for all x, y, which forces $R^{\top}R = I$. Geometrically: the columns are the body axes, which are orthonormal.
(2) Inverse is the transpose. From $R^{\top}R = I$. Read via (2.4): $(R_{AB})^{\top}$ has entries $b_i \cdot a_j$, i.e. it is "A's axes written in B", which is R_{BA}. So $R_{BA} = R_{AB}^{-1} = R_{AB}^{\top}$: inverting a rotation costs nothing.

(3) Determinant $+1$. $\det(R^\top R) = (\det R)^2 = 1$. A physical rotation can be reached continuously from the identity (just turn the body slowly). det is continuous and can only take the values ± 1, so it cannot jump: it stays at $\det I = 1$. Equivalently $\det R = b_x \cdot (b_y \times b_z) = +1$ says the body frame is right-handed.

Hence

$$\boxed{SO(3) = \{R \in \mathbb{R}^{3\times 3} : \; R^\top R = I, \; \det R = 1\}}$$

is a **group**: closed under products ($(R_1R_2)^\top R_1R_2 = I$, $\det = 1\cdot 1$), has an identity, has inverses ($R^\top$), is associative, but is *not* commutative (§2.1.6). It is *not a vector space*: $R + R = 2R$ has determinant 8 and stretches vectors, and $\frac{1}{2}(R_1 + R_2)$ is generally not a rotation. Averaging, interpolating or adding small corrections therefore needs the tools of §2.4.

2.2.5 Composition, and the order of multiplication

Chaining frames is cancellation of inner indices: ${}^A p = R_{AB}\,{}^B p = R_{AB}R_{BC}\,{}^C p \Rightarrow R_{AC} = R_{AB}R_{BC}$. When you instead *apply an extra rotation* Δ to a body with current orientation R, it matters which frame Δ is described in:

Derivation: pre-multiply = fixed axes, post-multiply = body axes

World axes. Rotating the body by Δ about world-fixed axes rotates each column (each body axis): $R_{\text{new}} = \Delta R$. *Body axes.* "Rotate by Δ about the body's own axes" means: the axis vector k given in body coordinates is Rk in world coordinates. By (2.3), rotation about Rk is the conjugate $R\Delta R^\top$. Applying that on the left: $R_{\text{new}} = (R\Delta R^\top)R = R\Delta$.

Rule: pre-multiplication rotates about the fixed (world) axes; post-multiplication rotates about the current (body) axes. This single fact explains intrinsic vs. extrinsic Euler angles in §2.3 and left vs. right perturbations in §2.4.

2.2.6 Relative orientation

Given two orientations in a common frame, the rotation between them follows from index cancellation:

$$R_{BC} = R_{BA}R_{AC} = R_{AB}^\top R_{AC}.$$

The "size" of this relative rotation (the angle needed to align the frames) is computed in §2.4 by $\theta = \arccos\big((\operatorname{tr} R_{BC} - 1)/2\big)$.

2.2.7 Degrees of freedom: why 3, not 9

Nine entries, but $R^\top R = I$ is symmetric, so it gives 6 independent scalar equations (3 unit lengths, 3 perpendicularities). $9 - 6 = 3$. Constructive count: choose the direction of b_x (a point on the unit sphere: 2 DOF), then b_y in the circle perpendicular to it (1 DOF); $b_z = b_x \times b_y$ is forced. Any representation with more than 3 numbers carries constraints (quaternion: 1, matrix: 6). Any representation with exactly 3 numbers must have a singularity somewhere (a topological fact about $SO(3)$ that §2.3 makes concrete).

2.2.8 Rotation vs. change of coordinates

The same matrix answers two questions. Let $R = R_z(90^\circ)$ and $p = (1,0,0)$. Active: rotating the vector p by 90° about z gives $(0,1,0)$. Passive: if frame $\{B\}$ is $\{A\}$ turned 90° about z, the point whose B-coordinates are $(1,0,0)$ has A-coordinates $(0,1,0)$. Same arithmetic, two stories. Write the frames as subscripts and you never need to remember which story a library intended: check that inner indices match.

Worked example: camera mounted on a robot (frame conventions)

A mobile robot uses the ROS body convention (x forward, y left, z up). Its camera uses the optical convention (z_C forward along the optical axis, x_C right in the image, y_C down). The camera looks straight ahead. Find R_{BC}, then convert a detection ${}^C p = (0.2, -0.1, 3.0)$ m into the body frame.
Columns = camera axes in body coordinates: x_C (right) $= -y_B \to (0,-1,0)$; y_C (down) $= -z_B \to (0,0,-1)$; z_C (forward) $= x_B \to (1,0,0)$:

$$R_{BC} = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}, \qquad \det R_{BC} = +1 \;\checkmark, \qquad {}^B p = R_{BC}\,{}^C p = (3.0, -0.2, 0.1).$$

Sanity check: 3 m ahead, 0.2 m to the right ($y_B < 0$), 0.1 m up (the detection was above the image centre, $y_C < 0$). Writing this matrix by inspection of columns is far less error-prone than composing "-90° about z then -90° about x", and it is the first thing to check when a perception pipeline produces mirrored or swapped detections.

Worked example: relative orientation of two IMUs (knee angle), part 1

Two IMUs are strapped to a thigh (1) and a shank (2). Each fusion filter reports its orientation relative to a common world frame. The physiotherapist wants the knee angle, which must not depend on which way the person is facing. With $R_{W1} = R_z(40°)R_y(10°)$ and $R_{W2} = R_{W1}R_y(55°)R_x(3°)$ (numerically, rounded):

$$R_{W1} = \begin{bmatrix} 0.7544 & -0.6428 & 0.1330 \\ 0.6330 & 0.7660 & 0.1116 \\ -0.1736 & 0 & 0.9848 \end{bmatrix}, \quad R_{12} = R_{W1}^{\top} R_{W2} = \begin{bmatrix} 0.5736 & 0.0429 & 0.8180 \\ 0 & 0.9986 & -0.0523 \\ -0.8192 & 0.0300 & 0.5728 \end{bmatrix}.$$

The heading 40° has cancelled exactly, as it must: R_{12} is a relation between the two segments only. We will extract the knee angle and hinge axis from R_{12} in §2.4.7 (Log map). Note the common mistake: subtracting Euler angles reported by the two filters. It works only by luck when the rotations share an axis.

Pitfalls in practice

- **Transpose confusion.** If an object appears to rotate the "wrong way" or a transform chain only works at zero angle, you used R_{BA} where R_{AB} was needed. Check: the columns should be the axes of the frame named *second*.
- **Drift off $SO(3)$.** After many floating-point products, $R^{\top}R \neq I$ slowly. Repair by projecting to the nearest rotation: with SVD $M = U\Sigma V^{\top}$, use $U\,\mathrm{diag}(1,1,\det(UV^{\top}))\,V^{\top}$. Better: integrate in a way that never leaves $SO(3)$ (§2.4.8).
- **Mirrored mounts.** A matrix built from hand-measured axes with $\det = -1$ means one axis is flipped. Always check the determinant.

2.3 Three angles and gimbal lock

2.3.1 The idea: build any rotation from three elementary turns

Since $SO(3)$ has three degrees of freedom, it is natural to try three numbers: perform three elementary rotations about coordinate axes. Consecutive axes must differ (two turns about the same axis merge into one), which leaves 12 sequences:

- **Tait–Bryan** (three distinct axes): $xyz, xzy, yxz, yzx, zxy, zyx$. Roll–pitch–yaw belongs here.
- **Proper Euler** (first axis repeated): $xyx, xzx, yxy, yzy, zxz, zyz$. Used for precession–nutation–spin and for robot wrists (§2.3.6).

On top of the sequence, each can be *intrinsic* (about the moving body axes) or *extrinsic* (about the fixed axes), giving 24 conventions in use. A list of three angles is meaningless until the convention is stated.

2.3.2 ZYX yaw–pitch–roll, intrinsic and extrinsic

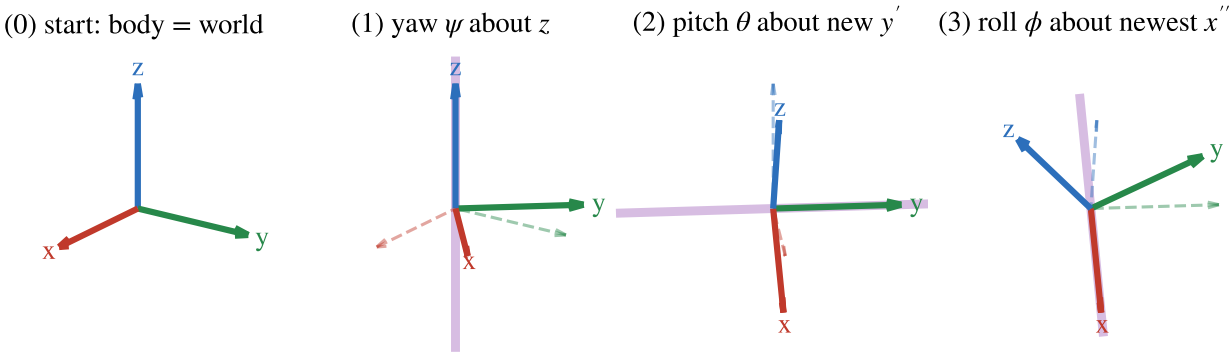


Figure 2.4: Intrinsic z–y'–x'': yaw about the world z, then pitch about the *new* y', then roll about the *newest* x''. Dashed: previous frame; purple: the axis used in that step.

Start aligned with the world. Yaw by ψ about z: orientation $R_z(\psi)$. Pitch by θ about the *current* y axis: by the post-multiplication rule (§2.2.5), $R_z(\psi)R_y(\theta)$. Roll by ϕ about the current x axis:

$$R(\psi,\theta,\phi) = R_z(\psi)\,R_y(\theta)\,R_x(\phi). \tag{2.5}$$

Read the same product right-to-left with the pre-multiplication rule: first $R_x(\phi)$ about world x, then $R_y(\theta)$ about world y,

then $R_z(\psi)$ about world z. Hence

$$\textbf{intrinsic } z\text{–}y'\text{–}x'' \;\equiv\; \textbf{extrinsic } x\text{–}y\text{–}z.$$

The angles are the same; only the story differs. Libraries name these differently (SciPy uses uppercase letters for intrinsic and lowercase for extrinsic sequences), which is the source of countless sign bugs.

Derivation: the ZYX matrix

Multiplying out (2.5):

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}. \tag{2.6}$$

The first column is $R_z(\psi)R_y(\theta)e_x$: the nose direction. It does not involve roll, as expected (rolling spins the body about its nose).

2.3.3 Recovering the angles

From (2.6): $r_{31} = -s_\theta$; $r_{21}/r_{11} = \tan\psi$; $r_{32}/r_{33} = \tan\phi$ (when $c_\theta \neq 0$). Robust formulas use atan2, which knows the quadrant:

$$\theta = \operatorname{atan2}\big(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2}\big), \qquad \psi = \operatorname{atan2}(r_{21}, r_{11}), \qquad \phi = \operatorname{atan2}(r_{32}, r_{33}).$$

There are always two solutions, (ψ, θ, ϕ) and $(\psi + \pi,\ \pi - \theta,\ \phi + \pi)$; restricting $\theta \in [-\pi/2, \pi/2]$ picks one. All three formulas divide by (or take the angle of a vector scaled by) c_θ. When $c_\theta \to 0$ both atan2 calls receive $(0,0)$ and become meaningless. That is the algebraic face of gimbal lock. Robust code detects $|r_{31}| \approx 1$ and then fixes one angle by convention (say $\psi = 0$) and recovers the other from the surviving combination: at $\theta = +90°$, $\phi - \psi = \operatorname{atan2}(r_{12}, r_{22})$; at $\theta = -90°$, $\phi + \psi = \operatorname{atan2}(-r_{12}, r_{22})$ (read off the matrix in §2.3.4 and its $-90°$ analogue). Without this branch, the generic formulas return $\operatorname{atan2}(0,0) = 0$ for both angles, which reconstructs a *different* rotation (practice problem 6).

2.3.4 Gimbal lock

Derivation: at pitch $90°$ only $\phi - \psi$ survives

Set $\theta = 90°$ ($s_\theta = 1,\ c_\theta = 0$) in (2.6) and use angle-difference identities:

$$R(\psi, 90°, \phi) = \begin{bmatrix} 0 & c_\psi s_\phi - s_\psi c_\phi & c_\psi c_\phi + s_\psi s_\phi \\ 0 & s_\psi s_\phi + c_\psi c_\phi & s_\psi c_\phi - c_\psi s_\phi \\ -1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \sin(\phi - \psi) & \cos(\phi - \psi) \\ 0 & \cos(\phi - \psi) & -\sin(\phi - \psi) \\ -1 & 0 & 0 \end{bmatrix}.$$

The matrix depends on ψ and ϕ only through $\phi - \psi$. Every pair with the same difference gives the *same* orientation. For example $(\psi, \phi) = (30°, 50°)$, $(0°, 20°)$ and $(-40°, -20°)$ produce identical matrices (checked numerically). At $\theta = -90°$ the same happens with $\phi + \psi$. The map from angles to rotations has collapsed a whole line of angle-triples onto one rotation, so it cannot be inverted there, and nearby it is badly conditioned.

Geometric picture. The three rotation axes of the parameterisation are: world z (yaw), $R_z(\psi)e_y$ (pitch), and $R_z(\psi)R_y(\theta)e_x$ (roll). At $\theta = \pm 90°$, $R_y(\theta)e_x = \mp e_z$, so the roll axis becomes parallel to the yaw axis. Three "knobs" now span only a 2-D set of instantaneous rotations; one direction of angular velocity (perpendicular to both the yaw and pitch axes) cannot be produced by any finite angle rates (Fig. 2.5).

What gimbal lock is not. The body is not physically stuck: it can still rotate in any direction. What fails is the *coordinate system* (and a mechanical gimbal that implements it). The same thing happens with latitude/longitude at the North Pole: you can walk in any direction, but longitude is undefined there, and small steps can change it by $180°$. This is unavoidable: no 3-number parameterisation covers all of $SO(3)$ smoothly and one-to-one (a topological fact). You can only move the singularity, never remove it.

2.3.5 Euler-angle rates are not angular velocity

Derivation: the map $(\dot\phi, \dot\theta, \dot\psi) \mapsto \omega_b$ and its determinant

The body angular velocity is defined by $[\omega_b]_\times = R^\top \dot R$ (derived in §2.4.8). Differentiate $R = R_z R_y R_x$ with the product rule, using $\dot R_z = R_z[\dot\psi e_z]_\times$ etc.:

$$R^\top \dot R = R_x^\top R_y^\top [\dot\psi e_z]_\times R_y R_x + R_x^\top [\dot\theta e_y]_\times R_x + [\dot\phi e_x]_\times.$$

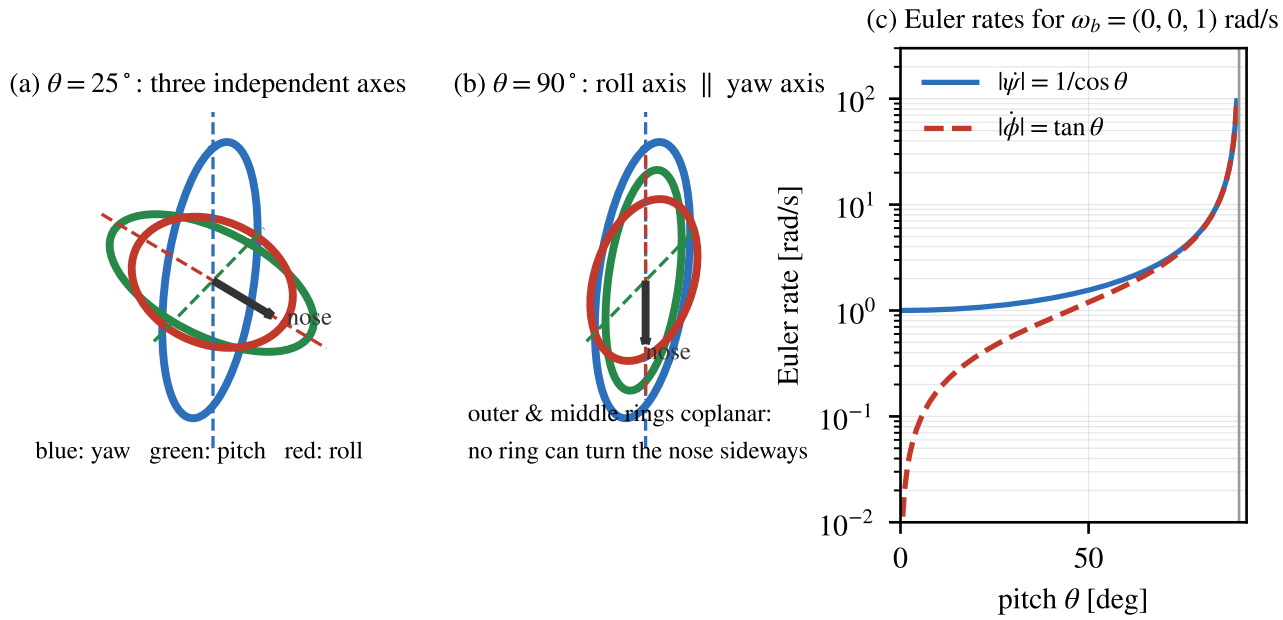


Figure 2.5: A mechanical ZYX gimbal: blue ring turns about the world z (yaw), green about the yaw-ring's y (pitch), red about the pitch-ring's x (roll), and the body (black "nose") is fixed in the red ring. (a) Generic pose: the three rotation axes are independent. (b) At $\theta = 90^\circ$ the roll axis lines up with the yaw axis and the outer two rings become coplanar; turning either one produces the same motion of the nose, and there is no ring left that can move the nose sideways. (c) To produce a modest body yaw rate of 1 rad/s, the required Euler rates diverge as $\theta \to 90^\circ$.

By (2.3), $R^\top [a]_\times R = [R^\top a]_\times$, so $\omega_b = \dot{\phi}\, e_x + \dot{\theta}\, R_x^\top e_y + \dot{\psi}\, R_x^\top R_y^\top e_z$. Evaluating the two rotated unit vectors:

$$\omega_b = \underbrace{\begin{bmatrix} 1 & 0 & -s_\theta \\ 0 & c_\phi & s_\phi c_\theta \\ 0 & -s_\phi & c_\phi c_\theta \end{bmatrix}}_{E(\phi,\theta)} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}, \qquad \det E = c_\theta .$$

E is singular exactly at $\theta = \pm 90^\circ$. Away from it,

$$\dot{\phi} = \omega_x + (s_\phi \omega_y + c_\phi \omega_z)\tan\theta, \qquad \dot{\theta} = c_\phi \omega_y - s_\phi \omega_z, \qquad \dot{\psi} = \frac{s_\phi \omega_y + c_\phi \omega_z}{\cos\theta}.$$

(Checked against finite differences.) The angle rates are the components of ω along three *non-orthogonal* axes. Only when the angles are all small is $E \approx I$, so that angle rates $\approx$ gyro readings.

2.3.6 Practical consequences

- **Discontinuities.** Yaw wraps at $\pm 180^\circ$. Averaging two headings 179° and -179° arithmetically gives 0°, which is 180° wrong. Controllers and filters with Euler states need angle-wrapping everywhere.
- **Interpolation.** Interpolating the three angles linearly produces a path in $SO(3)$ that is neither shortest nor constant-speed (Fig. 2.10: 232° of turning to achieve a 153° reorientation).
- **Control and estimation.** Near the singularity, small attitude errors become huge angle errors, Jacobians blow up, and an EKF with Euler states diverges. Aircraft that hover nose-up (tail-sitters), aerobatic drones, and humanoid torsos all reach $\pm 90^\circ$ pitch.
- **When Euler angles are fine:** human-readable display and logs, specifying set-points, and systems that never come near the singularity (ground vehicles, ships). Store and compute with something else; convert to Euler only at the edges.

Worked example: a tail-sitter VTOL in hover: tiny attitude change, huge angle change

A tail-sitter drone hovers nose-up, so its ZYX pitch is near 90°. True attitude $(\psi, \theta, \phi) = (10^\circ, 89.5^\circ, 20^\circ)$. The estimator's attitude is off by only 0.3°, a body-frame error $R_{\text{est}} = R_{\text{true}}\,\mathrm{Exp}(\delta)$. Extracted angles:

error δ (0.3° about body…)	ψ	θ	ϕ
none	10.00°	89.50°	20.00°
x (the nose)	10.00°	89.50°	20.30°
z	35.07°	89.33°	45.07°
y	35.20°	89.76°	45.20°

A 0.3° attitude error moved yaw and roll by 25° each, while $\phi - \psi = -10°$ stayed put, exactly as the gimbal-lock derivation predicts. A yaw controller fed these numbers would command a violent correction for an error that does not exist. Fix used in practice: control on $SO(3)$ directly (error $= \mathrm{Log}(R_d^\top R)$), or re-define the body frame so that hover corresponds to zero pitch.
Rates. At $\theta = 80°$, $\phi = 0$, a gentle body yaw rate $\omega_b = (0,0,1)$ rad/s corresponds to $\dot\psi = 1/\cos 80° = 5.76$ rad/s and $\dot\phi = \tan 80° = 5.67$ rad/s (Fig. 2.5c). An Euler-angle integrator needs a tiny time step here.

Worked example: the spherical wrist of a 6-DOF arm is a ZYZ gimbal

Many industrial arms (e.g. most KUKA, ABB and Fanuc models) end in three revolute joints whose axes meet at one point. Their orientation is $R_{\text{wrist}} = R_z(\theta_4)R_y(\theta_5)R_z(\theta_6)$: a proper Euler ZYZ sequence. At $\theta_5 = 0$ joints 4 and 6 are coaxial: only $\theta_4 + \theta_6$ matters, inverse kinematics has infinitely many solutions, and keeping the tool moving along certain directions requires $\dot\theta_4, \dot\theta_6 \to \infty$. Robot programmers call this the *wrist singularity*; it is gimbal lock in hardware, and motion planners avoid paths with $\theta_5 \approx 0$ for exactly the reason in Fig. 2.5(c).

2.4 Angle–axis and exponential coordinates

2.4.1 Euler's rotation theorem: every rotation has an axis

Derivation: $R \in SO(3)$ always fixes a line

We need a vector n with $Rn = n$, i.e. $\det(R - I) = 0$. Using $RR^\top = I$ and $\det R = \det R^\top = 1$:

$$\det(R-I) = \det(R - RR^\top) = \det R \det(I - R^\top) = \det\left((I-R)^\top\right) = \det(I-R) = (-1)^3 \det(R-I).$$

So $\det(R-I) = -\det(R-I)$, hence it is 0: eigenvalue 1 exists, with a unit eigenvector $\hat n$. The plane perpendicular to $\hat n$ is mapped to itself (dot products are preserved), and there R acts as a 2×2 orthogonal matrix of determinant 1, which is a planar rotation by some θ. Therefore every 3-D rotation is "turn by θ about one fixed axis $\hat n$". (The odd dimension is what makes this work. In 4-D a rotation can turn two planes independently with no fixed axis at all.)

2.4.2 Axis–angle and its ambiguities

The pair $(\hat n, \theta)$ with $\|\hat n\| = 1$ is the most geometric description possible: *which* line stays fixed, and *how much* everything turns around it. It uses four numbers with one constraint. Its ambiguities are exactly the places where it stops being one-to-one:

- $(\hat n, \theta)$, $(-\hat n, -\theta)$ and $(\hat n, \theta + 2\pi k)$ are the same rotation; restrict to $\theta \in [0, \pi]$.
- $\theta = 0$: the axis is undefined (every axis works for "no rotation").
- $\theta = \pi$: $(\hat n, \pi)$ and $(-\hat n, \pi)$ coincide (turning half a turn either way lands at the same place).

2.4.3 The cross product as a matrix

Because $\omega \times v$ is linear in v:

$$\omega \times v = [\omega]_\times v, \qquad [\omega]_\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}, \qquad [\omega]_\times^\top = -[\omega]_\times.$$

The algebra of rotations follows from (2.1). For unit $\hat n$: $[\hat n]_\times^2 v = \hat n \times (\hat n \times v) = \hat n(\hat n \cdot v) - v$, so

$$[\hat n]_\times^2 = \hat n \hat n^\top - I, \qquad [\hat n]_\times^3 = -[\hat n]_\times, \qquad [\hat n]_\times^4 = -[\hat n]_\times^2, \ \ldots \tag{2.7}$$

Geometrically: $\hat n \hat n^\top v$ is the component of v along the axis, $-[\hat n]_\times^2 v$ is the perpendicular component, and $[\hat n]_\times v$ is the perpendicular component turned 90°. Applying $[\hat n]_\times$ twice turns it 180° (a minus sign), which is why the powers cycle exactly like those of the imaginary unit i.

2.4.4 Rodrigues' formula, derived geometrically

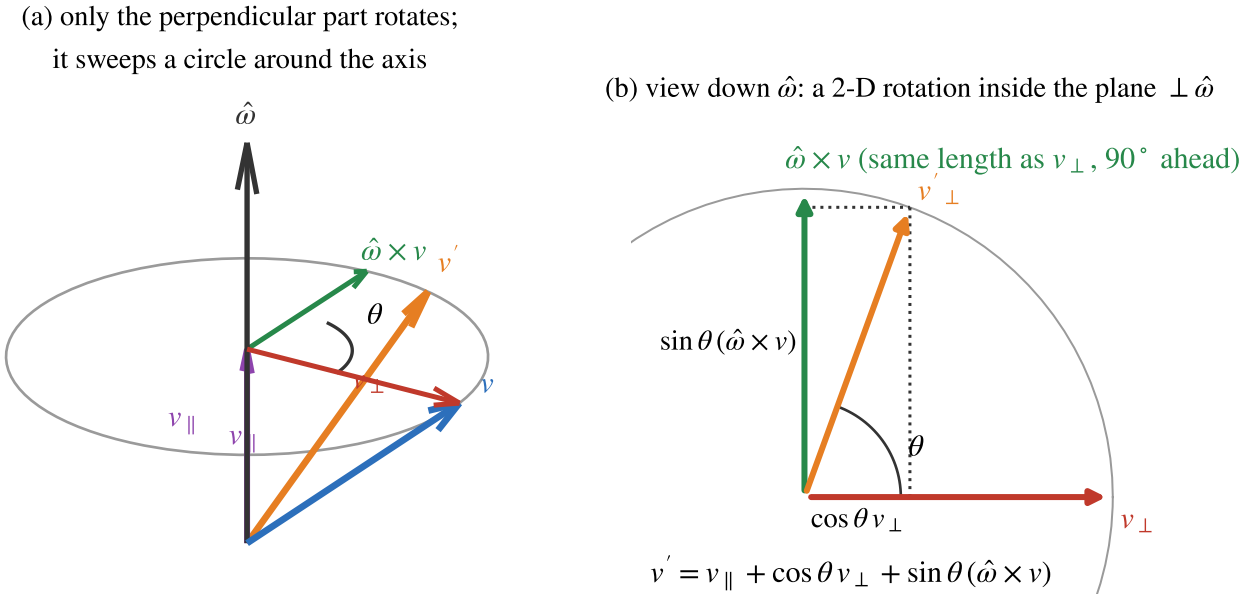


Figure 2.6: Rodrigues' construction. (a) Split v into the part along the axis (unchanged by the rotation) and the part perpendicular to it (which sweeps a circle). (b) In the perpendicular plane, $v_\perp$ and $\hat{\omega} \times v$ form an orthogonal pair of equal length, so turning by θ is "$\cos\theta$ of one plus $\sin\theta$ of the other", just as in 2-D.

Derivation: rotation by θ about $\hat{n}$

Decompose $v = v_\parallel + v_\perp$ with $v_\parallel = (\hat{n} \cdot v)\hat{n} = (I + [\hat{n}]_\times^2)v$ and $v_\perp = -[\hat{n}]_\times^2 v$. The parallel part does not move. The perpendicular part lives in a plane where $\{v_\perp,\ \hat{n} \times v\}$ is an orthogonal pair of equal length ($\hat{n} \times v = \hat{n} \times v_\perp$). A planar rotation by θ gives

$$v' = v_\parallel + \cos\theta\, v_\perp + \sin\theta\,(\hat{n} \times v) = (I + [\hat{n}]_\times^2)v - \cos\theta\,[\hat{n}]_\times^2 v + \sin\theta\,[\hat{n}]_\times v.$$

Collecting terms gives Rodrigues' rotation formula:

$$\boxed{R(\hat{n},\theta) = I + \sin\theta\,[\hat{n}]_\times + (1 - \cos\theta)\,[\hat{n}]_\times^2} \tag{2.8}$$

2.4.5 The exponential map: rotation as "spinning for a while"

Where does an exponential come from? From physics. Spin a body at unit rate about the fixed axis $\hat{n}$. Every point moves with velocity $\dot{p} = \hat{n} \times p = [\hat{n}]_\times p$, a linear ODE with constant coefficients. Its solution is $p(t) = e^{[\hat{n}]_\times t} p(0)$. After time θ the body has turned by angle θ, so

$$R(\hat{n},\theta) = e^{[\hat{n}]_\times \theta}.$$

Derivation: Rodrigues = matrix exponential

Expand the series and use the cycling powers (2.7) to replace $[\hat{n}]_\times^3 \to -[\hat{n}]_\times$, $[\hat{n}]_\times^4 \to -[\hat{n}]_\times^2$, etc.:

$$e^{[\hat{n}]_\times \theta} = I + \underbrace{\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \cdots\right)}_{\sin\theta}[\hat{n}]_\times + \underbrace{\left(\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \cdots\right)}_{1-\cos\theta}[\hat{n}]_\times^2,$$

which is exactly (2.8). The two derivations meet: the geometric one (decompose and rotate) and the dynamical one (integrate a constant spin). Numerically, (2.8) and `scipy.linalg.expm` agree to 10^{-16}.

Exponential coordinates. Pack axis and angle into one 3-vector, the *rotation vector* $\varphi = \theta\hat{n} \in \mathbb{R}^3$ (direction = axis, length = angle). Then

$$\mathrm{Exp}(\varphi) = e^{[\varphi]_\times} = I + \frac{\sin\|\varphi\|}{\|\varphi\|}[\varphi]_\times + \frac{1 - \cos\|\varphi\|}{\|\varphi\|^2}[\varphi]_\times^2.$$

Three numbers, no constraint, smooth, and $\mathrm{Exp}(0) = I$. The ball $\|\varphi\| \le \pi$ covers every rotation once, except that antipodal surface points $\pm\pi\hat{n}$ are the same rotation. Because of that gluing of the boundary, no 3-number chart can be globally smooth and one-to-one; this is the deeper reason behind gimbal lock in §2.3.

2.4.6 The Lie algebra $\mathfrak{so}(3)$: the space of infinitesimal rotations

Derivation: tangent vectors to $SO(3)$ are skew-symmetric

Let $R(t)$ be any smooth path of rotations. Differentiate $R^\top R = I$: $\dot{R}^\top R + R^\top \dot{R} = 0$, i.e. $R^\top \dot{R} = -(R^\top \dot{R})^\top$. So $R^\top \dot{R}$ is always skew-symmetric. At $R(0) = I$ the velocity $\dot{R}(0)$ itself is skew. The tangent space at the identity is therefore

$$\mathfrak{so}(3) = \{\Omega \in \mathbb{R}^{3\times 3} : \Omega^\top = -\Omega\} = \{[\omega]_\times : \omega \in \mathbb{R}^3\} \cong \mathbb{R}^3.$$

$\mathfrak{so}(3)$ is a genuine vector space: you can add, scale and average elements of it. The exponential maps it onto the group; the logarithm maps (a neighbourhood of) the group back. Non-commutativity survives only infinitesimally through the bracket $[[a]_\times, [b]_\times] = [a]_\times[b]_\times - [b]_\times[a]_\times = [a \times b]_\times$. That is why rotation vectors add only to first order:

$$\mathrm{Log}\big(\mathrm{Exp}(a)\,\mathrm{Exp}(b)\big) = a + b + \tfrac{1}{2}\,a \times b + O(\|a\|, \|b\|)^3 \qquad \text{(BCH formula).} \tag{2.9}$$

Example: $a = (0.3, 0, 0)$, $b = (0, 0.4, 0)$ gives $\mathrm{Log}(\mathrm{Exp}\,a\,\mathrm{Exp}\,b) = (0.296, 0.397, 0.060)$, while $a + b = (0.3, 0.4, 0)$ and $a + b + \frac{1}{2}a \times b = (0.3, 0.4, 0.06)$.

2.4.7 The logarithm: from a matrix back to axis and angle

Derivation: trace and skew part of Rodrigues

Take the trace of (2.8): $\mathrm{tr}\,I = 3$, $\mathrm{tr}[\hat{n}]_\times = 0$, $\mathrm{tr}[\hat{n}]_\times^2 = \mathrm{tr}(\hat{n}\hat{n}^\top - I) = 1 - 3 = -2$. So $\mathrm{tr}\,R = 1 + 2\cos\theta$. Take the skew part: $R - R^\top = 2\sin\theta\,[\hat{n}]_\times$ (the $[\hat{n}]_\times^2$ term is symmetric). Hence

$$\theta = \arccos\frac{\mathrm{tr}\,R - 1}{2} \in [0, \pi], \qquad [\hat{n}]_\times = \frac{R - R^\top}{2\sin\theta}, \qquad \varphi = \mathrm{Log}(R) = \theta\hat{n}.$$

Numerical care. (i) Near $\theta = 0$, compute $\theta/\sin\theta$ with its Taylor series $1 + \theta^2/6$ instead of dividing two tiny numbers; likewise $\sin\theta/\theta \approx 1 - \theta^2/6$ and $(1 - \cos\theta)/\theta^2 \approx \frac{1}{2} - \theta^2/24$ in Exp. (ii) Near $\theta = \pi$, $\sin\theta \to 0$ and the skew part vanishes. Use the symmetric part instead: $\frac{1}{2}(R + R^\top) = \cos\theta\, I + (1 - \cos\theta)\hat{n}\hat{n}^\top$, so $\hat{n}\hat{n}^\top = \big(\frac{1}{2}(R + R^\top) - \cos\theta I\big)/(1 - \cos\theta)$; take the column with the largest diagonal entry and normalise. The sign of $\hat{n}$ is genuinely ambiguous at exactly π. (iii) Clamp $(\mathrm{tr}\,R - 1)/2$ to $[-1, 1]$ before arccos; floating point will occasionally give 1.0000000002.

Worked example: knee angle and hinge axis, part 2

From part 1 (§2.2), $\mathrm{tr}\,R_{12} = 2.1450$, so $\theta = \arccos(0.5725) = 55.08°$, and Log gives axis $\hat{n} = (0.050, 0.998, -0.026)$. The joint turned $55.1°$ about an axis within about $3°$ of the shank's y-axis, which is the anatomical flexion axis. The small off-axis part is the $3°$ ab/adduction. This single number is frame-independent: unlike "difference of pitch angles", it does not change if the person turns around or if the IMUs are mounted rolled. Gait-analysis toolkits report joint angles this way, or decompose R_{12} with a joint-specific Euler sequence chosen so that the singularity is outside the physiological range.

2.4.8 Angular velocity and the kinematics of rotation

For a moving body $R(t) = R_{WB}(t)$, both $\dot{R}R^\top$ and $R^\top\dot{R}$ are skew (§2.4.6), so define

$$\dot{R} = [\omega_s]_\times R \quad \text{(spatial / world angular velocity)}, \qquad \dot{R} = R[\omega_b]_\times \quad \text{(body angular velocity)}.$$

They are the same physical spin expressed in two frames: by (2.3), $[\omega_s]_\times = R[\omega_b]_\times R^\top = [R\omega_b]_\times$, so $\omega_s = R\,\omega_b$. *Physical meaning:* a body point with fixed body coordinates ${}^B p$ is at $p(t) = R(t)\,{}^B p$ in the world, and $\dot{p} = \dot{R}\,{}^B p = [\omega_s]_\times p = \omega_s \times p$: the familiar "$v = \omega \times r$". A gyroscope bolted to the body measures ω_b.

Exact integration. If ω_b is constant over a step Δt, then $\dot{R} = R[\omega_b]_\times$ has the exact solution

$$R_{k+1} = R_k\,\mathrm{Exp}(\omega_b\,\Delta t).$$

Each factor is exactly a rotation, so the product stays on $SO(3)$ forever, up to round-off.

An honest caveat from the same simulation: the *attitude* errors (after re-projecting the naive estimate onto $SO(3)$) were about 0.3–0.5° for all three methods after 120 s. That error is dominated by treating ω as constant during each step, not

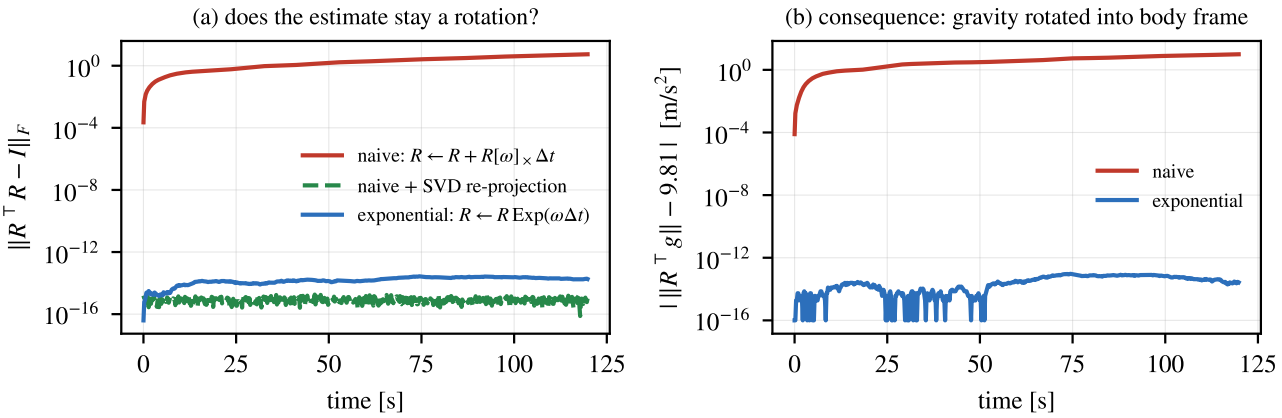


Figure 2.7: Integrating a 100 Hz gyro signal for two minutes. (a) The naive Euler step $R \leftarrow R(I + [\omega]_\times \Delta t)$ leaves $SO(3)$: each factor has singular values $\sqrt{1 + \|\omega\|^2 \Delta t^2} > 1$, so errors compound geometrically. The exponential update stays orthogonal to machine precision. (b) Why it matters: an IMU must subtract gravity rotated into the body frame. With the naive matrix the rotated gravity vector has the wrong length, so the navigation filter sees phantom accelerations: about 0.01 m/s^2 after 1 s and ≈ 10 m/s^2 after 2 minutes.

by the parameterisation. The exponential map removes the structural error (leaving the group). It does not fix sampling or sensor errors; for those, use higher-order schemes (e.g. midpoint ω, or pre-integration with coning correction) and a filter.

2.4.9 Perturbations, errors and optimisation on $SO(3)$

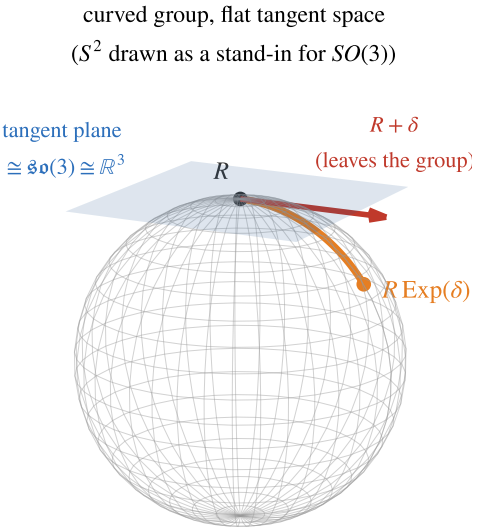


Figure 2.8: Adding a small vector to a point on a curved surface leaves the surface; walking along the surface in that direction (the exponential) does not. Estimators therefore keep the state R on the group and represent *corrections* as tangent vectors $\delta \in \mathbb{R}^3$: $R \boxplus \delta = R\,\mathrm{Exp}(\delta)$ (body / right perturbation) or $\mathrm{Exp}(\delta)\,R$ (world / left perturbation). The difference between two rotations is the tangent vector $\delta = \mathrm{Log}(R_1^\top R_2)$. Both conventions are valid; mixing them inside one Jacobian is a classic bug.

Derivation: the Jacobian behind every SLAM / VIO residual

How does a rotated point Rp change under a small body-frame correction δ? Use $\mathrm{Exp}(\delta) \approx I + [\delta]_\times$:

$$R\,\mathrm{Exp}(\delta)p \approx Rp + R(\delta \times p) = Rp - R[p]_\times \delta \quad \Longrightarrow \quad \frac{\partial\,(R\,\mathrm{Exp}(\delta)p)}{\partial \delta}\Big|_{\delta=0} = -R[p]_\times .$$

A Gauss–Newton step then solves for $\delta \in \mathbb{R}^3$ (an ordinary vector), and updates $R \leftarrow R\,\mathrm{Exp}(\delta)$. No constraints, no singularities, and the state is always a valid rotation. This "minimal tangent update, full group state" pattern is the core of GTSAM, Ceres manifolds, Sophus/manif, and error-state Kalman filters.

Worked example: measuring rotation error correctly (trajectory evaluation)

Ground truth $R_g = \mathrm{Exp}(0.1, -0.2, 0.3)$, estimate $R_e = \mathrm{Exp}(0.12, -0.18, 0.29)$ (rotation vectors in rad). The geodesic error is

$$e = \mathrm{Log}(R_g^\top R_e) = (0.0214,\ 0.0162,\ -0.0130)\ \text{rad}, \qquad \|e\| = 1.711^\circ .$$

Subtracting the rotation vectors gives $(0.02, 0.02, -0.01)$: almost the same length ($1.719°$) but pointing about $10°$ away, since (2.9) only holds to first order. For large rotations the naive difference is meaningless. Two useful identities: the angle can be read from $\operatorname{tr}(R_g^\top R_e) = 1 + 2\cos\|e\|$, and the Frobenius ("chordal") distance is $\|R_g - R_e\|_F = 2\sqrt{2}\sin(\|e\|/2)$ (here 0.04223, matching). Benchmarks such as the TUM and KITTI odometry metrics compute rotation errors in exactly this geodesic way.

Worked example: averaging rotations (extrinsic calibration)

Three calibration runs estimate the camera-to-IMU rotation, nominally $R_0 = R_z(90°)R_x(-90°)$, as $R_i = R_0\operatorname{Exp}(\delta_i)$ with $\delta_i = (2,-1,0.5)°,\ (-1,1.5,1)°,\ (0.5,-0.5,-2)°$. What is "the average"?
Element-wise mean of matrices: $M = \frac{1}{3}\sum R_i$ has $\det M = 0.99866$ and $\|M^\top M - I\|_F = 1.7 \times 10^{-3}$. It is not a rotation (it shrinks vectors), and for widely spread inputs it degrades badly.
Intrinsic (Karcher) mean: find $\bar{R}$ with $\sum_i \operatorname{Log}(\bar{R}^\top R_i) = 0$, i.e. the tangent-space errors balance. Iterate

$$\delta \leftarrow \tfrac{1}{N}\textstyle\sum_i \operatorname{Log}(\bar{R}^\top R_i), \qquad \bar{R} \leftarrow \bar{R}\operatorname{Exp}(\delta).$$

Starting from $\bar{R} = R_1$, the step sizes were $1.92°$, $1.2 \times 10^{-4}°$, $8 \times 10^{-9}°$: converged in two iterations, to $\bar{R} = R_0\operatorname{Exp}\big((0.500,\ 0,\ -0.167)°\big)$. For tightly clustered data this equals the mean of the δ_i, and also the SVD projection of M onto $SO(3)$ ("chordal mean"). The pattern "map to tangent space, do Euclidean arithmetic, map back" is how every statistic (mean, covariance, interpolation, filtering) is done on $SO(3)$.

2.5 Unit quaternions

2.5.1 Why another representation?

Rotation matrices: 9 numbers, 6 constraints, 27 multiplications to compose, and re-orthonormalisation is awkward. Euler angles: singular. Rotation vectors: minimal and great for corrections, but composing two of them needs Exp, a product and Log (with trigonometry each time). Quaternions sit in the sweet spot: 4 numbers, *one* easy constraint ($\|q\| = 1$, fixed by dividing by the norm), composition by a 16-multiplication product, no singularities, and a natural geometry for interpolation and averaging. That is why IMU firmware, flight controllers, game engines, ROS `tf`, and most SLAM front-ends store attitude as a quaternion.

2.5.2 Quaternion algebra from one rule

Hamilton's rule $i^2 = j^2 = k^2 = ijk = -1$ defines everything. Right-multiplying $ijk = -1$ by k gives $ij(-1) = -k$, so $ij = k$; similarly $jk = i$, $ki = j$, and reversing the order flips the sign ($ji = -k$, etc.). These are exactly the cross-product rules for e_x, e_y, e_z, with an extra -1 for squares.

Write $q = w + xi + yj + zk = (w, u)$ with scalar part w and vector part $u = (x, y, z)$. For two *pure* quaternions (zero scalar part), expanding with the rules above gives the key identity

$$(0,a)(0,b) = (-a\cdot b,\ a\times b). \tag{2.10}$$

The quaternion product packages the dot and cross products into one operation. By bilinearity, the general product is

$$\boxed{(w_1,u_1)\otimes(w_2,u_2) = \big(w_1w_2 - u_1\cdot u_2,\ \ w_1u_2 + w_2u_1 + u_1\times u_2\big)}$$

It is associative but not commutative (because of the $u_1 \times u_2$ term), just like rotations.

Conjugate, norm, inverse. $q^* = (w, -u)$; $q \otimes q^* = (w^2 + \|u\|^2, 0) = \|q\|^2$; $q^{-1} = q^*/\|q\|^2$. The norm is multiplicative: $\|p \otimes q\|^2 = pqq^*p^* = \|q\|^2\|p\|^2$. So unit quaternions (the 3-sphere $S^3 \subset \mathbb{R}^4$) are closed under multiplication, and for them $q^{-1} = q^*$. The analogy to $SO(3)$ is exact: unit norm $\leftrightarrow$ orthogonality, conjugate $\leftrightarrow$ transpose.

2.5.3 How a unit quaternion rotates a vector, and where the half-angle comes from

Embed a vector v as the pure quaternion $(0, v)$ and define v' by the *sandwich product* $(0, v') = q \otimes (0, v) \otimes q^*$.

Derivation: the sandwich is a rotation, and why $\theta/2$

Step 1: it is a rotation. The result is pure: $\operatorname{Re}(qvq^*) = \operatorname{Re}(vq^*q) = \operatorname{Re}(v) = 0$ (using $\operatorname{Re}(ab) = \operatorname{Re}(ba)$). It preserves length: $\|qvq^*\| = \|v\|$. It is linear in v and equals the identity at $q = 1$. A length-preserving linear map connected to

the identity is a rotation.
Step 2: which rotation. Write $q = (\cos\alpha,\ \sin\alpha\,\hat{n}) = \cos\alpha + \sin\alpha\,\hat{n}$ and split $v = v_\parallel + v_\perp$ relative to $\hat{n}$. From (2.10): $\hat{n}\,v_\parallel = -\hat{n}\cdot v_\parallel + 0 = v_\parallel\,\hat{n}$ (they *commute*), while $\hat{n}\,v_\perp = \hat{n}\times v_\perp = -v_\perp\,\hat{n}$ (they *anticommute*). Therefore

$$q\,v_\parallel\,q^* = v_\parallel\,q\,q^* = v_\parallel \qquad \text{(the two factors cancel)},$$
$$q\,v_\perp\,q^* = q\,q\,v_\perp = q^2\,v_\perp \qquad \text{(the two factors add)},$$

where $v_\perp q^* = v_\perp(\cos\alpha - \sin\alpha\,\hat{n}) = (\cos\alpha + \sin\alpha\,\hat{n})v_\perp = q\,v_\perp$ was used. Now $q^2 = \cos 2\alpha + \sin 2\alpha\,\hat{n}$ (de Moivre, since $\hat{n}^2 = -1$), so

$$q\,v_\perp\,q^* = \cos 2\alpha\,v_\perp + \sin 2\alpha\,(\hat{n}\times v_\perp).$$

This is Rodrigues' formula with rotation angle $\theta = 2\alpha$. To rotate by θ we must take

$$\boxed{q(\hat{n},\theta) = \left(\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\hat{n}\right)}$$

The half angle is there because q acts twice (once from each side). On the perpendicular component the two half-turns add up to a full θ; on the parallel component they cancel. A one-sided product $q\otimes(0,v)$ rotates $v_\perp$ by only $\theta/2$ and turns $v_\parallel$ into a quaternion with a non-zero scalar part, so the result is not even a vector in $\mathbb{R}^3$.

Expanding the sandwich in components gives the explicit form $v' = (w^2 - \|u\|^2)v + 2(u\cdot v)u + 2w\,(u\times v)$.

2.5.4 Composition

Since $q_1(q_2 v q_2^*)q_1^* = (q_1q_2)v(q_1q_2)^*$, quaternions compose in the same order as matrices: $q_{AC} = q_{AB}\otimes q_{BC}$ and $R(q_1\otimes q_2) = R(q_1)R(q_2)$. Pre-multiplication = world axes, post-multiplication = body axes, exactly as in §2.2.5.

Worked example: non-commutativity, computed exactly

$q_z = (\frac{\sqrt{2}}{2},0,0,\frac{\sqrt{2}}{2})$ is 90° about z; $q_x = (\frac{\sqrt{2}}{2},\frac{\sqrt{2}}{2},0,0)$ is 90° about x. With the product formula:

$$q_z\otimes q_x = \big(\tfrac12,\ \underbrace{\tfrac12(1,0,0)+\tfrac12(0,0,1)+\tfrac12(0,1,0)}_{w_1u_2+w_2u_1+u_1\times u_2}\big) = \tfrac12(1,1,1,1), \qquad q_x\otimes q_z = \tfrac12(1,1,-1,1).$$

Reading off $\cos(\theta/2) = \frac12$: both are 120° rotations, about $(1,1,1)/\sqrt{3}$ and $(1,-1,1)/\sqrt{3}$ respectively; these are the two final boxes in Fig. 2.2. Check by sandwich: q_zq_x sends $e_y \mapsto e_z$ (R_x takes $y\to z$, then R_z leaves z alone). Note the symmetric axis $(1,1,1)$: a 120° turn about it cyclically permutes $x\to y\to z$.

2.5.5 Conversions

Quaternion → matrix. Apply the sandwich to e_x, e_y, e_z and use $w^2+x^2+y^2+z^2 = 1$:

$$R(q) = \begin{bmatrix} 1-2(y^2+z^2) & 2(xy-wz) & 2(xz+wy) \\ 2(xy+wz) & 1-2(x^2+z^2) & 2(yz-wx) \\ 2(xz-wy) & 2(yz+wx) & 1-2(x^2+y^2) \end{bmatrix}.$$

Matrix → quaternion. From the diagonal: $\operatorname{tr}R = 3-4(x^2+y^2+z^2) = 4w^2-1$, so $w = \frac12\sqrt{1+\operatorname{tr}R}$, then $x = (r_{32}-r_{23})/4w$, $y = (r_{13}-r_{31})/4w$, $z = (r_{21}-r_{12})/4w$. This divides by w, which is tiny for rotations near 180°. *Shepperd's method* fixes that: compute whichever of w^2,x^2,y^2,z^2 is largest from the diagonal (e.g. $4x^2 = 1+2r_{11}-\operatorname{tr}R$) and get the other three from off-diagonal sums/differences, dividing by the largest. Every serious library does this.

Rotation vector ↔ quaternion. $q = \operatorname{Exp}(\varphi) = \big(\cos\frac{\|\varphi\|}{2},\ \sin\frac{\|\varphi\|}{2}\,\frac{\varphi}{\|\varphi\|}\big)$ and $\varphi = 2\,\operatorname{atan2}(\|u\|,w)\,u/\|u\|$. The atan2 form is accurate at all angles, unlike $\arccos w$ near $w=\pm1$.

2.5.6 Double cover: q and $-q$

$R(q)$ is quadratic in q, so $R(-q) = R(q)$: every rotation has exactly two unit quaternions, at opposite points of S^3. Physically, $-q = (\cos\frac{\theta+2\pi}{2}, \sin\frac{\theta+2\pi}{2}\hat{n})$ is "the same rotation plus one extra full turn". Fig. 2.9 shows the consequence: as a body turns continuously about a fixed axis, its matrix returns to I after 2π but its quaternion only after 4π. The 3-sphere S^3 is the double cover of $SO(3)$, and unlike $SO(3)$ it has no "glued boundary", which is why quaternions have no singularity. **Practical rules.** (1) The angle between two orientations is $2\arccos|q_1\cdot q_2|$ (note the absolute value). (2) Before interpolating or blending, flip one quaternion if $q_1\cdot q_2 < 0$. (3) To average many quaternions, do not sum them; use

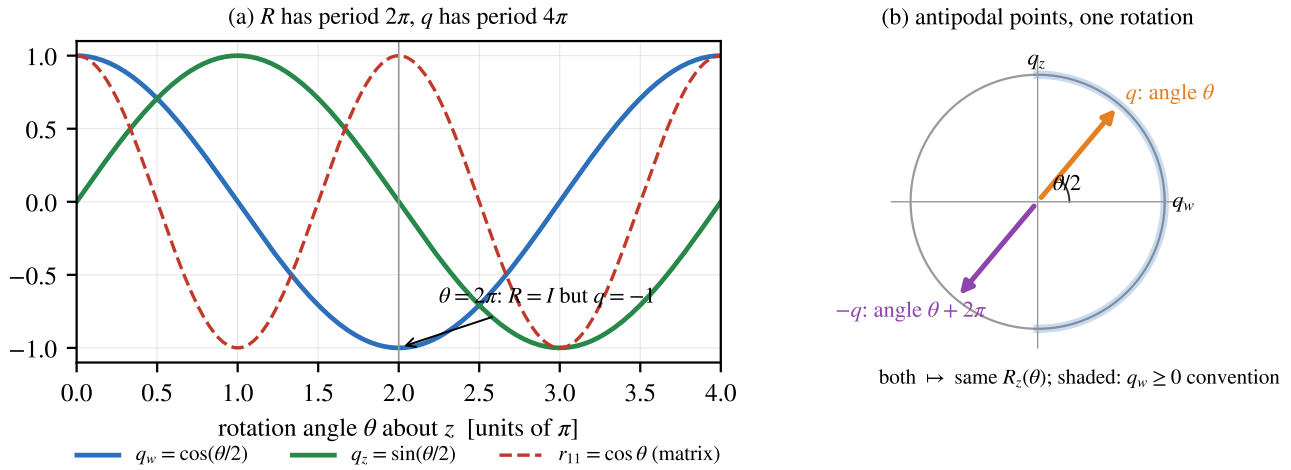


Figure 2.9: (a) Quaternion components for a rotation by θ about z, compared with a rotation-matrix entry. (b) In the (q_w, q_z) plane, q sits at angle $\theta/2$; $-q$ is diametrically opposite and encodes the same rotation.

the eigenvector of $\sum_i q_i q_i^\top$ with the largest eigenvalue (Markley's method), which is insensitive to signs. (4) In a Kalman filter, a measured quaternion may arrive with the opposite sign to the state; compute the innovation as $\mathrm{Log}(q_{\text{state}}^* \otimes q_{\text{meas}})$ after enforcing a non-negative scalar part on the product.

2.5.7 Quaternion kinematics and IMU integration

With $\dot{R} = R[\omega_b]_\times$ and $q(t+\Delta t) = q(t) \otimes \mathrm{Exp}(\omega_b \Delta t) \approx q \otimes (1, \tfrac{1}{2}\omega_b \Delta t)$:

$$\dot{q} = \tfrac{1}{2}\, q \otimes (0, \omega_b) = \tfrac{1}{2}\,(0, \omega_s) \otimes q, \qquad q_{k+1} = q_k \otimes \left(\cos \frac{\|\omega\|\Delta t}{2},\ \sin \frac{\|\omega\|\Delta t}{2}\, \frac{\omega}{\|\omega\|} \right).$$

The $\frac{1}{2}$ is the half-angle again. Renormalise q occasionally to remove round-off (cheap and harmless, unlike orthonormalising a matrix).

Worked example: integrating a gyro with quaternions

A gyro reports constant $\omega_b = (0.4, -0.2, 1.0)$ rad/s at 200 Hz. After 200 steps (1 s) of the update above, $q = (0.8537,\ 0.1901,\ -0.0951,\ 0.4754)$, identical to $\mathrm{Exp}(\omega_b \cdot 1\,\mathrm{s})$ to all printed digits: for constant rate the exponential update is exact, regardless of step size. Sanity check: $\|\omega_b\| = 1.095$ rad, and $\cos(1.095/2) = 0.8537$.

2.5.8 Interpolation: LERP, NLERP and SLERP

To move smoothly from q_0 to q_1 (camera keyframes, animation, trajectory set-points) we want the shortest rotation, traversed at constant angular speed. On $SO(3)$ that is $q(t) = q_0 \otimes \mathrm{Exp}\left(t\,\mathrm{Log}(q_0^* \otimes q_1)\right)$: compute the relative rotation, take a fraction t of its rotation vector, apply it.

Derivation: the SLERP formula

On S^3 the shortest path is a great-circle arc. Let $\cos\Omega = q_0 \cdot q_1$ (after flipping q_1 if negative). The unit vector in the plane of q_0, q_1 orthogonal to q_0 is $q_\perp = (q_1 - \cos\Omega\, q_0)/\sin\Omega$. Walking an angle $t\Omega$ along the arc:

$$q(t) = \cos(t\Omega)\, q_0 + \sin(t\Omega)\, q_\perp = \frac{\sin((1-t)\Omega)\, q_0 + \sin(t\Omega)\, q_1}{\sin\Omega}.$$

The physical rotation angle between q_0 and q_1 is 2Ω (half-angle once more). For $\Omega \to 0$ switch to normalised linear interpolation to avoid $0/0$.

Worked example: the sign check matters: yaw across $\pm 180°$

A ship's heading goes from $170°$ to $-170°$ (a $20°$ turn through south). $q_a = (0.0872, 0, 0, 0.9962)$. Converting $R_z(-170°)$ with a library that enforces $w \geq 0$ gives $q_b = (0.0872, 0, 0, -0.9962)$, and $q_a \cdot q_b = -0.985$. SLERP without the sign check returns the midpoint $(1, 0, 0, 0)$: heading $0°$, i.e. it turned $340°$ the long way. With the check ($q_b \to -q_b$) the midpoint is $(0, 0, 0, 1)$: heading $180°$, correct.

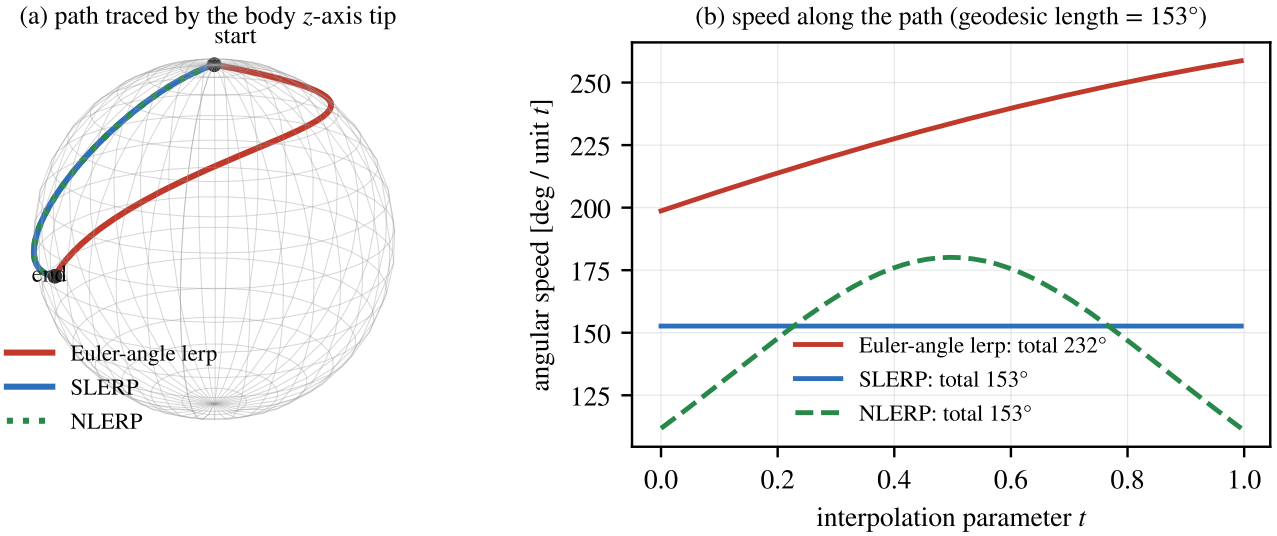


Figure 2.10: Interpolating from identity to ZYX $(150°, 50°, -120°)$. (a) SLERP and NLERP follow the same great-circle path (they overlap); linear interpolation of Euler angles wanders. (b) SLERP has constant angular speed; NLERP (normalised linear interpolation of q) has the same path but speeds up in the middle; Euler-angle interpolation turns 232° in total to achieve a 153° reorientation.

Worked example: rolling-shutter / timestamp interpolation

A camera's rows are exposed at different times; a row captured 40% of the way between two IMU-propagated poses needs the orientation at $t = 0.4$. Take q_0 = the quaternion of ZYX $(20°, 5°, -3°) = (0.9833, -0.0333, 0.0384, 0.1745)$ and q_1 = that of ZYX $(32°, 2°, 1°) = (0.9611, 0.0036, 0.0192, 0.2754)$. Then $\Omega = 6.38°$ (a 12.77° rotation) and $q(0.4) = (0.9759, -0.0186, 0.0308, 0.2152)$, i.e. ZYX $(24.83°, 3.90°, -1.32°)$; its geodesic distance from q_0 is exactly 0.4 of the total. Interpolating the Euler angles linearly would give $(24.8°, 3.8°, -1.4°)$: close here only because the rotation is small and far from the singularity, and wrong by degrees for fast motions.

2.5.9 Strengths, limitations and conventions

\+ Singularity-free, compact, cheap to compose and renormalise; smooth interpolation and averaging; exact integration for constant rates.

– Not minimal (4 numbers, 1 constraint): an EKF must not put q directly in its error state (the 4×4 covariance would be singular). Error-state filters keep q as the nominal state and a 3-vector $\delta\theta$ as the error, $q = \hat{q} \otimes \mathrm{Exp}(\delta\theta)$.

– Double cover: sign ambiguity in comparisons, interpolation, averaging and filtering.

! **Conventions.** Hamilton ($ij = k$, used here, ROS, Eigen, most of robotics) vs. JPL ($ij = -k$, common in older aerospace and some VIO papers): the product formulas differ in the sign of the cross term. Storage order (w, x, y, z) vs. (x, y, z, w) differs between libraries, and mixing them silently produces wrong rotations without crashing.

2.6 3-D pose and screw motion

2.6.1 From orientation to pose

A rigid body has 6 degrees of freedom: 3 for where its frame's origin is, 3 for how the frame is turned. The pose of $\{B\}$ in $\{A\}$ is the pair (R_{AB}, t_{AB}), where t_{AB} is the origin of B written in A-coordinates.

Derivation: how a point's coordinates change between frames

The arrow from A's origin to a point p equals (arrow from A's origin to B's origin) + (arrow from B's origin to p). In A-coordinates the first is t_{AB}; the second is known in B-coordinates and converted with R_{AB}:

$$ {}^{A}p = R_{AB}\,{}^{B}p + t_{AB}. $$

Directions (differences of two points) lose the t_{AB}: ${}^{A}d = R_{AB}\,{}^{B}d$.

2.6.2 Homogeneous coordinates and the transformation matrix

The map above is affine, not linear, so it does not compose by plain matrix products. Append one coordinate: 1 for points, 0 for directions. Then

$$\begin{bmatrix} {}^{A}p \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R_{AB} & t_{AB} \\ 0 & 1 \end{bmatrix}}_{T_{AB}} \begin{bmatrix} {}^{B}p \\ 1 \end{bmatrix}, \qquad \begin{bmatrix} {}^{A}d \\ 0 \end{bmatrix} = T_{AB} \begin{bmatrix} {}^{B}d \\ 0 \end{bmatrix}.$$

The trailing 0 makes directions immune to translation automatically, which is the point/vector distinction of §2.1.1 built into the arithmetic. The set

$$SE(3) = \left\{ T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} : R \in SO(3),\ t \in \mathbb{R}^3 \right\}$$

is a group (the special Euclidean group): every rigid motion preserves distances and handedness, and every such map of space has this form.

Derivation: composition and inverse

Multiplying blocks: $T_{AB}T_{BC} = \begin{bmatrix} R_{AB}R_{BC} & R_{AB}t_{BC} + t_{AB} \\ 0 & 1 \end{bmatrix} = T_{AC}$. Read the translation part: "C's origin in B, rotated into A, plus B's origin in A". For the inverse, solve ${}^{A}p = R\,{}^{B}p + t$ for ${}^{B}p$:

$$T_{AB}^{-1} = T_{BA} = \begin{bmatrix} R^{\top} & -R^{\top}t \\ 0 & 1 \end{bmatrix}.$$

The term $-R^{\top}t$ is A's origin written in B. A common bug is to write $-t$ instead of $-R^{\top}t$, which is correct only when $R = I$.

Frame-notation rules. (1) T_{AB} maps B-coordinates to A-coordinates *and* is the pose of B in A. (2) Adjacent indices must match and cancel: $T_{WC} = T_{WB}T_{BC}$; a product like $T_{WB}T_{CB}$ is a type error. (3) Relative pose between two keyframes: $T_{12} = T_{W1}^{-1}T_{W2}$. (4) If a motion Δ is known in frame C but you need it in frame B, conjugate: $\Delta_B = T_{BC}\Delta_C T_{BC}^{-1}$. Hand–eye calibration $AX = XB$ is exactly this equation with the unknown mount $X = T_{BC}$.

World → base → camera → object: ${}^{W}p = T_{WB}\,T_{BC}\,{}^{C}p$

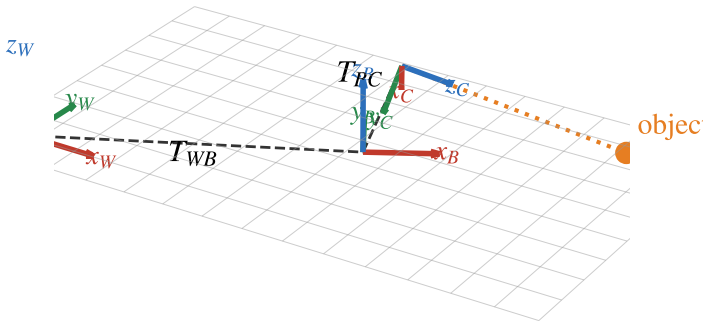


Figure 2.11: A chain of frames: the robot base $\{B\}$ in the world, a camera $\{C\}$ on the robot (pitched down 20°), and an object seen by the camera. Composing the transforms along the chain maps the camera measurement into world coordinates.

Worked example: camera detection to world map, and back to pixels

Given. Robot at $(2.0, 1.0, 0)$ m, yaw 30°: $T_{WB} = \begin{bmatrix} R_z(30°) & (2,1,0)^{\top} \\ 0 & 1 \end{bmatrix}$. Camera at $(0.25, 0, 0.6)$ in the base frame,

optical convention, pitched down 20°: $R_{BC} = R_y(20^\circ)\,R^0_{BC}$ with R^0_{BC} from the example in §2.2, giving

$$R_{BC} = \begin{bmatrix} 0 & -0.342 & 0.940 \\ -1 & 0 & 0 \\ 0 & -0.940 & -0.342 \end{bmatrix}, \qquad T_{WC} = T_{WB}T_{BC} = \begin{bmatrix} 0.500 & -0.296 & 0.814 & 2.217 \\ -0.866 & -0.171 & 0.470 & 1.125 \\ 0 & -0.940 & -0.342 & 0.600 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Forward. A detection at ${}^{C}p = (0.3, -0.1, 1.5)$ gives ${}^{B}p = T_{BC}{}^{C}p = (1.694, -0.300, 0.181)$ and ${}^{W}p = (3.617, 1.587, 0.181)$: 18 cm above the floor, as it should be for an object seen slightly above the optical axis by a down-tilted camera.
Backward (map landmark → pixel). $T_{CW} = T_{WC}^{-1}$ (use the $R^\top, -R^\top t$ formula). A mapped landmark ${}^{W}l = (4.0, 2.5, 0.3)$ is at ${}^{C}l = (-0.299, -0.482, 2.200)$. With pinhole intrinsics $f = 600\,\text{px}$, $(c_u, c_v) = (320, 240)$: $u = 320 + 600 \cdot (-0.299/2.2) = 238.4$, $v = 240 + 600 \cdot (-0.482/2.2) = 108.7$. This "transform, then project" is the forward model inside every visual localisation and bundle-adjustment residual.

2.6.3 Estimating a pose from point correspondences (Kabsch / Umeyama)

Derivation: closed-form least-squares rigid alignment

Given model points p_i and measured points $q_i \approx Rp_i + t$, minimise $\sum_i \|q_i - Rp_i - t\|^2$. Setting the derivative w.r.t. t to zero: $t = \bar{q} - R\bar{p}$ (centroids must align). With centred points $p'_i = p_i - \bar{p}$, $q'_i = q_i - \bar{q}$, the cost becomes $\text{const} - 2\sum_i (q'_i)^\top R p'_i = \text{const} - 2\,\text{tr}(RH)$ with $H = \sum_i p'_i (q'_i)^\top$. Take the SVD $H = USV^\top$; then $\text{tr}(RH) = \text{tr}\left((V^\top RU)S\right)$ and $V^\top RU$ is orthogonal, so the trace is largest when $V^\top RU = I$:

$$R = V\,\text{diag}\big(1, 1, \det(VU^\top)\big)\,U^\top, \qquad t = \bar{q} - R\bar{p}.$$

The det term prevents returning a reflection when the data are noisy or nearly planar.

Worked example: object pose from five 3-D points

A 20 cm object has 5 known model points; a depth camera measures them with 2 mm noise. Kabsch recovers the pose with rotation error 1.30° and translation error 2.2 mm (RMS residual 3.8 mm). Note the large rotation error: angular accuracy is roughly noise/size, so for small objects orientation is the weak part of the estimate. The same routine is the inner step of ICP, aligns estimated and ground-truth trajectories before computing ATE, and (with a scale factor, Umeyama) aligns monocular SLAM trajectories.

2.6.4 Chasles' theorem: every rigid displacement is a screw

Derivation: rotation about some axis + translation along the same axis

Take (R, t) with R a rotation by $\theta \neq 0$ about $\hat{n}$. Split $t = t_\parallel + t_\perp$ along and across $\hat{n}$. A screw motion (rotate by θ about the line through point q with direction $\hat{n}$, then slide d along it) maps $p \mapsto R(p-q) + q + d\hat{n} = Rp + (I-R)q + d\hat{n}$. Match with $Rp + t$:

$$(I - R)q + d\hat{n} = t.$$

Dotting with $\hat{n}$ (note $\hat{n}^\top(I-R) = 0$ because $R^\top\hat{n} = \hat{n}$) gives $d = \hat{n}\cdot t$. The remaining equation $(I-R)q = t_\perp$ is solvable because on the plane perpendicular to $\hat{n}$, R is a rotation by $\theta \neq 0$ and has no fixed vector, so $I - R$ is invertible there. Hence *any* rigid displacement is a screw: rotation θ about a unique axis, plus translation d along that same axis. Its **pitch** is $h = d/\theta$ (metres per radian). Pure rotation: $h = 0$. Pure translation: $\theta = 0$, the limiting case $h = \infty$.

2.6.5 Twists: the velocity of a rigid body

Differentiate a moving pose $T(t)$, as we did for $R(t)$:

$$T^{-1}\dot{T} = \begin{bmatrix} R^\top\dot{R} & R^\top\dot{t} \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} [\omega_b]_\times & v_b \\ 0 & 0 \end{bmatrix}, \qquad \dot{T}T^{-1} = \begin{bmatrix} \dot{R}R^\top & \dot{t} - \dot{R}R^\top t \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} [\omega_s]_\times & v_s \\ 0 & 0 \end{bmatrix}.$$

Both have the form $[\xi] = \begin{bmatrix} [\omega]_\times & v \\ 0 & 0 \end{bmatrix} \in \mathfrak{se}(3)$, the Lie algebra of $SE(3)$, identified with the **twist** $\xi = (v, \omega) \in \mathbb{R}^6$.

- **Body twist** ξ_b: ω_b and $v_b = R^\top\dot{t}$, the velocity of the body origin, both in body coordinates. This is what wheel odometry and a VIO front-end produce.

- **Spatial twist ξ_s**: ω_s and $v_s = \dot{t} - \omega_s \times t$. v_s is the velocity of the (imaginary) body point that is currently passing through the world origin. Strange, but it makes ξ_s independent of where the body frame is attached.

Screw axis → twist. A unit-speed screw with direction $\hat{n}$, a point q on the axis and pitch h has $\omega = \hat{n}$ and $v = -\hat{n} \times q + h\hat{n}$. The term $-\hat{n} \times q = \hat{n} \times (0 - q)$ is the velocity of the origin caused by spinning about an axis through q; $h\hat{n}$ is the slide along the axis. For a revolute joint $h = 0$; for a prismatic joint $\omega = 0$, $v = \hat{n}$.

2.6.6 The exponential of a twist

Derivation: $e^{[\xi]\theta}$ generates the screw motion

Let $\|\omega\| = 1$. Powers of the 4×4 matrix: $[\xi]^k = \begin{bmatrix} [\omega]_\times^k & [\omega]_\times^{k-1} v \\ 0 & 0 \end{bmatrix}$ for $k \geq 1$. Summing the series block-wise:

$$e^{[\xi]\theta} = \begin{bmatrix} e^{[\omega]_\times \theta} & G(\theta)v \\ 0 & 1 \end{bmatrix}, \qquad G(\theta) = \sum_{k \geq 1} \frac{\theta^k}{k!} [\omega]_\times^{k-1} = I\theta + (1 - \cos\theta)[\omega]_\times + (\theta - \sin\theta)[\omega]_\times^2,$$

where (2.7) collapses the series exactly as for Rodrigues. Now insert the screw twist $v = -\omega \times q + h\omega$:

- $G(\theta)(h\omega) = h\theta\,\omega$, because $[\omega]_\times \omega = 0$: the slide along the axis is $h\theta$.
- $G(\theta)(-[\omega]_\times q) = -\theta[\omega]_\times q - (1 - \cos\theta)[\omega]_\times^2 q - (\theta - \sin\theta)[\omega]_\times^3 q = -\sin\theta[\omega]_\times q - (1 - \cos\theta)[\omega]_\times^2 q = (I - e^{[\omega]_\times \theta})q$.

So $e^{[\xi]\theta} = \begin{bmatrix} R & (I-R)q + h\theta\,\omega \\ 0 & 1 \end{bmatrix}$, precisely the screw of Chasles' theorem: rotate about the line through q, slide $h\theta$ along it. Special cases: $h = 0$ (pure rotation about an off-origin axis, translation $(I - R)q$); $\omega = 0$, $\|v\| = 1$ (pure translation, $e^{[\xi]\theta} = \begin{bmatrix} I & v\theta \\ 0 & 1 \end{bmatrix}$).

For a general rotation vector $\varphi = \omega\theta$ and $\rho = v\theta$, the same result reads $\mathrm{Exp}(\rho, \varphi) = \begin{bmatrix} \mathrm{Exp}(\varphi) & V(\varphi)\rho \\ 0 & 1 \end{bmatrix}$ with $V = I + \frac{1-\cos\|\varphi\|}{\|\varphi\|^2}[\varphi]_\times + \frac{\|\varphi\| - \sin\|\varphi\|}{\|\varphi\|^3}[\varphi]_\times^2$. Note the coupling matrix V: *the translation of* Exp *is not* ρ. Exponentiating rotation and translation separately (a common shortcut) gives a different, frame-dependent motion. **Log map:** θ, ω from $\mathrm{Log}(R)$, then $v = G(\theta)^{-1} t$ with $G^{-1} = \frac{1}{\theta} I - \frac{1}{2}[\omega]_\times + \left(\frac{1}{\theta} - \frac{1}{2}\cot\frac{\theta}{2}\right)[\omega]_\times^2$ (or $v = t/\|t\|$, $\theta = \|t\|$ if $R = I$).

Worked example: screw parameters of a gripper motion, and how to interpolate it

A gripper moves from the identity pose to $T = \begin{bmatrix} R_z(90^\circ) & (1,1,0.5)^\top \\ 0 & 1 \end{bmatrix}$. By hand: the axis is along z; $d = \hat{n} \cdot t = 0.5$; solve $(I - R)q = (1,1,0)$ with $I - R_z(90^\circ) = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$ on the xy-block, giving $q = (0,1)$. So the motion is a 90° turn about the vertical line through $(0,1)$ plus a 0.5 m rise along it: pitch $h = 0.5/(\pi/2) = 0.318$ m/rad. The Log formula agrees: $\xi\theta = (1.571, 0, 0.5;\ 0, 0, 1.571)$, i.e. $\theta = \pi/2$, $\omega = e_z$, $v = (1, 0, 0.318)$, $\omega \cdot v = 0.318 = h$, $\omega \times v = (0,1,0) = q$.

Interpolation. $T(s) = \exp(s[\xi\theta])$ moves at constant twist along a helix. At $s = 0.5$: $R = R_z(45^\circ)$, $t = (0.707, 0.293, 0.25)$, which is the origin rotated 45° about $(0,1)$ and lifted 0.25. Fig. 2.12 compares this with the usual "lerp translation + slerp rotation". Both are valid paths with the same endpoints, but the screw path does not depend on which point of the gripper you call the origin: changing the tool frame by X gives exactly $T(s)X$. The decoupled path changes shape when the tool frame is moved, because it moves the chosen origin in a straight line and swings every other point around it.

2.6.7 Changing the frame of a twist: the adjoint

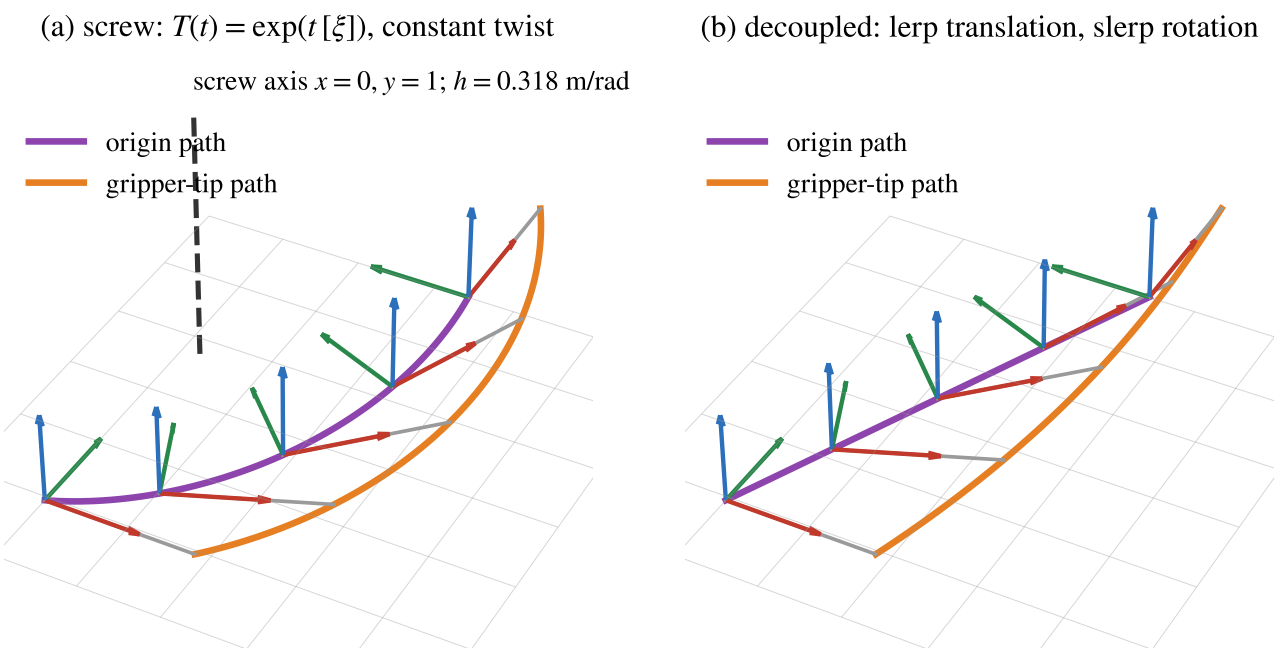


Figure 2.12: Interpolating the same displacement two ways. (a) Constant twist: every point follows a helix about one screw axis (dashed). (b) Decoupled: the frame origin moves on a straight line while the orientation slerps, so the gripper tip (orange) takes a different, origin-dependent path.

Derivation: $\xi_s = \mathrm{Ad}_T\,\xi_b$

From the two definitions, $[\xi_s] = \dot{T}T^{-1} = T(T^{-1}\dot{T})T^{-1} = T[\xi_b]T^{-1}$. Multiply out with (2.3):

$$T[\xi]T^{-1} = \begin{bmatrix} R[\omega]_\times R^\top & -R[\omega]_\times R^\top t + Rv \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} [R\omega]_\times & Rv + t \times R\omega \\ 0 & 0 \end{bmatrix} \Rightarrow \mathrm{Ad}_T = \begin{bmatrix} R & [t]_\times R \\ 0 & R \end{bmatrix} \quad \text{(ordering } (v, \omega)\text{)}.$$

The angular part simply rotates. The linear part rotates *and* picks up $t \times R\omega$: the lever-arm effect. The same rule moves a twist between any two frames: $\xi_A = \mathrm{Ad}_{T_{AB}}\,\xi_B$.

Worked example: what does the camera feel when the robot turns?

The base drives forward at 1 m/s while turning left at 0.5 rad/s: $\xi_B = (1, 0, 0;\ 0, 0, 0.5)$ (body twist). The forward-looking optical camera is mounted 0.3 m ahead and 0.5 m up (T_{BC} with R^0_{BC} from §2.2). Since $T_{WC} = T_{WB}T_{BC}$ with T_{BC} constant, the camera's body twist is $\xi_C = \mathrm{Ad}_{T_{CB}}\,\xi_B$, which gives

$$\xi_C = (-0.15,\ 0,\ 1.0;\ \ 0,\ -0.5,\ 0).$$

Reading it in camera axes: moving forward along the optical axis at 1 m/s, sliding *left* in the image at 0.15 m/s (lever arm 0.3 m $\times$ 0.5 rad/s, because the camera is ahead of the turning centre), and rotating about $-y_C$ (image "up") at 0.5 rad/s. A visual-odometry or motion-blur model that ignored the lever arm would attribute that sideways slip to a nonexistent lateral velocity.

2.6.8 Forward kinematics by the product of exponentials

Derivation: $T(\theta) = e^{[S_1]\theta_1} \cdots e^{[S_n]\theta_n} M$

Let M be the end-effector pose with all joints at zero, and S_i the screw axis of joint i expressed in the world frame *at that home configuration*. Move only the last joint: the tool undergoes a screw about S_n, so $T = e^{[S_n]\theta_n} M$. Now move joint $n-1$: it carries joint n and the tool rigidly, and its own axis has not been moved by joint n (joint n is downstream). So $T = e^{[S_{n-1}]\theta_{n-1}} e^{[S_n]\theta_n} M$. Induct to the base. No Denavit–Hartenberg frames are needed: only n screw axes and one home pose.

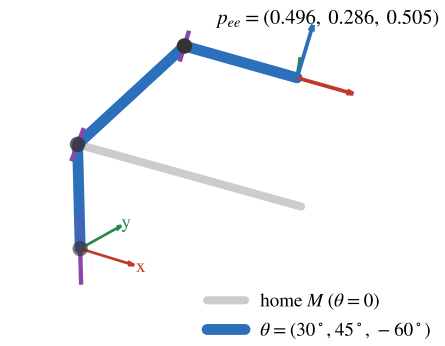


Figure 2.13: The 3-DOF elbow arm of the example at home (grey) and at $\theta = (30^\circ, 45^\circ, -60^\circ)$ (blue). Each joint is described only by its screw axis at home; the product of exponentials moves the tool.

Worked example: a 3-DOF elbow arm

Link lengths $L_0 = 0.3$ (base height), $L_1 = 0.4$, $L_2 = 0.3$ m; at home the arm points along $+x$. Joint 1: $\omega_1 = e_z$ through the origin. Joints 2, 3: $\omega = -e_y$ (positive angle lifts the arm) through $q_2 = (0, 0, L_0)$ and $q_3 = (L_1, 0, L_0)$. With $v_i = -\omega_i \times q_i$:

$$S_1 = (0,0,0;\,0,0,1),\ S_2 = (0.3,0,0;\,0,-1,0),\ S_3 = (0.3,0,-0.4;\,0,-1,0),$$

$M = \begin{bmatrix} I & (0.7,0,0.3)^\top \\ 0 & 1 \end{bmatrix}$. For $\theta = (30^\circ, 45^\circ, -60^\circ)$ the product gives tool position $(0.496, 0.286, 0.505)$ m and tool z-axis $(0.224, 0.129, 0.966)$. Elementary check: horizontal reach $L_1 \cos 45^\circ + L_2 \cos(-15^\circ) = 0.573$, height $L_0 + L_1 \sin 45^\circ + L_2 \sin(-15^\circ) = 0.505$, and $0.573(\cos 30^\circ, \sin 30^\circ) = (0.496, 0.286)$.

Velocity kinematics comes for free. The spatial Jacobian has columns $J_1 = S_1$ and $J_i = \mathrm{Ad}_{e^{[S_1]\theta_1 \cdots e^{[S_{i-1}]\theta_{i-1}}}} S_i$: each joint's screw axis, carried to its *current* location by the joints before it. For the arm above, $J_2 = (0.260, 0.150, 0;\ 0.5, -0.866, 0)$, $J_3 = (0.505, 0.291, -0.283;\ 0.5, -0.866, 0)$. For $\dot\theta = (0.1, -0.2, 0.3)$ rad/s, $\xi_s = J\dot\theta$ and the tool-tip velocity $v_s + \omega_s \times p$ is $(0.0271,\ 0.0818,\ -0.0276)$ m/s, which matches finite differences of the forward kinematics.

2.6.9 Pose optimisation: the $SE(3)$ version of "tangent update, group state"

Exactly as in §2.4.9, estimators keep $T \in SE(3)$ and solve for a 6-vector correction $\delta = (\rho, \varphi)$, updating $T \leftarrow T\,\mathrm{Exp}(\delta)$. Using $\mathrm{Exp}(\delta)p \approx p + \rho + \varphi \times p$:

$$\left.\frac{\partial\left(T\,\mathrm{Exp}(\delta)\,p\right)}{\partial \delta}\right|_{\delta=0} = R\left[\, I \;\middle|\; -[p]_\times \,\right] \in \mathbb{R}^{3\times 6}.$$

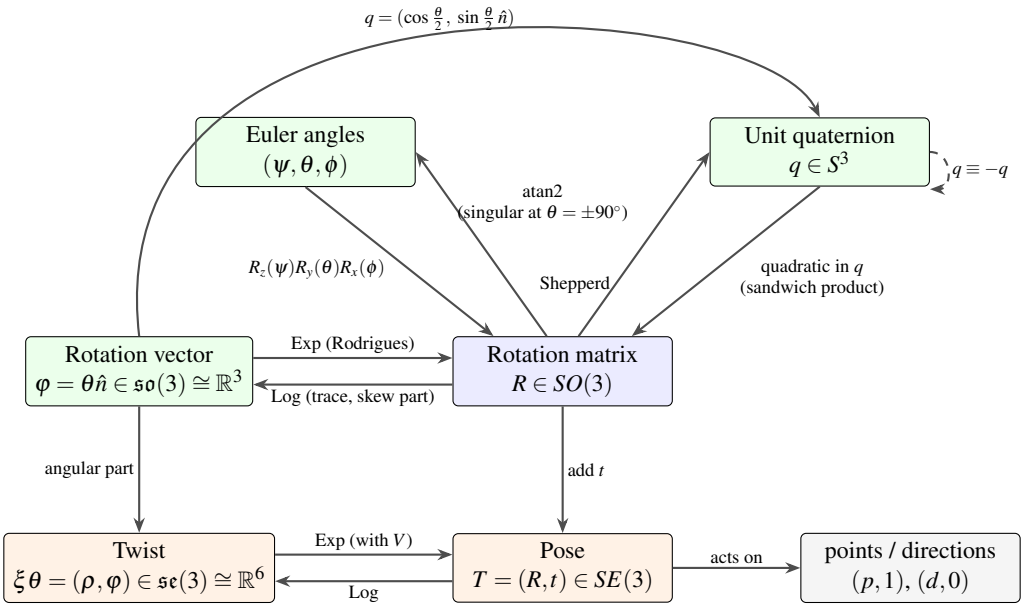
Chain this with the pinhole projection derivative and you have the analytic Jacobian of a reprojection error (bundle adjustment, PnP refinement); with a point-to-plane residual you have ICP. Twists, the adjoint and these Jacobians are the whole mathematical toolkit of modern SLAM, VIO, and robot calibration.

2.7 Putting it together

2.7.1 Comparing the representations

Representation	Numbers	Strength	Limitation	Use it for
Rotation matrix R	9 (6 constr.)	transforms vectors directly; composition = product; no ambiguity	redundant; drifts off $SO(3)$ numerically	applying rotations, transform chains, final computations
Euler / RPY angles	3	human-readable, minimal	singular at $\theta = \pm 90°$; wrap-around; convention zoo	display, logs, set-points, vehicles far from singularity
Axis–angle $(\hat{n}, \theta)$	4 (1 constr.)	clearest geometry; Euler's theorem	axis undefined at $\theta = 0$; $\pm$ at π	reasoning, joint axes, describing a relative rotation
Rotation vector φ	3	minimal; a vector space (tangent space)	Log ambiguous near π; does not add (BCH)	small corrections, errors, optimisation, filtering, averaging
Unit quaternion q	4 (1 constr.)	singularity-free; cheap composition; SLERP	$q \equiv -q$; not minimal; convention mix-ups	storing and propagating attitude, interpolation
Homogeneous T	16 (12 used)	composes poses by products; points and directions	redundant; inverse needs $R^\top$, $-R^\top t$	frame chains, kinematics, projection
Twist $\xi = (v, \omega)$	6	minimal; velocity and screw axis; vector space	Log ambiguities; frame (body/spatial) must be tracked	velocities, joint axes, PoE, pose optimisation

2.7.2 How everything connects



Green: parameterisations of $SO(3)$. Blue: the group itself (what actually acts on vectors). Orange: the same story one level up, for poses. Horizontal arrows on the left are Exp / Log between the flat tangent spaces and the curved groups; this is the bridge every estimator, controller and planner crosses.

2.7.3 Formula sheet

Skew matrix	$[\omega]_\times v = \omega \times v;\ [\hat{n}]_\times^2 = \hat{n}\hat{n}^\top - I;\ [\hat{n}]_\times^3 = -[\hat{n}]_\times;\ [R\omega]_\times = R[\omega]_\times R^\top$
Rodrigues / Exp	$R = I + \sin\theta[\hat{n}]_\times + (1-\cos\theta)[\hat{n}]_\times^2 = e^{[\hat{n}]_\times\theta}$
Log	$\theta = \arccos\frac{\operatorname{tr}R-1}{2},\ [\hat{n}]_\times = \frac{R-R^\top}{2\sin\theta}$; near π: $\hat{n}\hat{n}^\top = \frac{(R+R^\top)/2-\cos\theta I}{1-\cos\theta}$
Angular velocity	$\dot{R} = [\omega_s]_\times R = R[\omega_b]_\times,\ \omega_s = R\omega_b;$ exact step $R_{k+1} = R_k\operatorname{Exp}(\omega_b\Delta t)$
ZYX Euler	$R = R_z(\psi)R_y(\theta)R_x(\phi);\ \theta = \operatorname{atan2}(-r_{31}, \sqrt{r_{11}^2+r_{21}^2}),\ \psi = \operatorname{atan2}(r_{21},r_{11}),\ \phi = \operatorname{atan2}(r_{32},r_{33});\ \det E = \cos\theta$
Quaternion	$q = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2}\hat{n});\ (w_1,u_1)(w_2,u_2) = (w_1w_2 - u_1\cdot u_2,\ w_1u_2 + w_2u_1 + u_1\times u_2);\ v' = qvq^*;\ \dot{q} = \frac{1}{2}q\otimes(0,\omega_b);$ angle $= 2\arccos\lvert q_1\cdot q_2\rvert$
SLERP	$q(t) = \frac{\sin((1-t)\Omega)q_0 + \sin(t\Omega)q_1}{\sin\Omega},\ \cos\Omega = q_0\cdot q_1 \geq 0$ (flip if negative)
Pose	$T_{AB} = \begin{bmatrix} R_{AB} & t_{AB} \\ 0 & 1 \end{bmatrix};\ T_{AB}^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0 & 1 \end{bmatrix};\ T_{AC} = T_{AB}T_{BC}$
Twist	$[\xi] = \begin{bmatrix} [\omega]_\times & v \\ 0 & 0 \end{bmatrix}$; screw: $\omega = \hat{n},\ v = -\hat{n}\times q + h\hat{n};\ T^{-1}\dot{T} = [\xi_b],\ \dot{T}T^{-1} = [\xi_s]$
SE(3) Exp	$e^{[\xi]\theta} = \begin{bmatrix} e^{[\omega]_\times\theta} & (I\theta + (1-\cos\theta)[\omega]_\times + (\theta-\sin\theta)[\omega]_\times^2)v \\ 0 & 1 \end{bmatrix}$
Adjoint	$\operatorname{Ad}_T = \begin{bmatrix} R & [t]_\times R \\ 0 & R \end{bmatrix}$ for $\xi = (v,\omega);\ \xi_A = \operatorname{Ad}_{T_{AB}}\xi_B$
PoE kinematics	$T(\theta) = e^{[S_1]\theta_1}\cdots e^{[S_n]\theta_n}M;\ J_i = \operatorname{Ad}_{e^{[S_1]\theta_1}\cdots e^{[S_{i-1}]\theta_{i-1}}}S_i$
Perturbation Jacobians	$\partial(R\operatorname{Exp}(\delta)p)/\partial\delta = -R[p]_\times;\ \partial(T\operatorname{Exp}(\delta)p)/\partial\delta = R[I \mid -[p]_\times]$
Kabsch	$H = \sum p_i'(q_i')^\top = USV^\top,\ R = V\operatorname{diag}(1,1,\det VU^\top)U^\top,\ t = \bar{q} - R\bar{p}$

2.7.4 Self-assessment checklist

You understand this chapter if you can do each of these *without notes*, and say why it works:

1. Explain why 3-D orientation has exactly three degrees of freedom (§2.2.7), and why every 3-number chart has a singularity (§2.4.5).
2. Write R_{AB} from a description of axes by columns, and compose/invert chains without index mistakes (§2.2.2, §2.2.5).
3. Derive the ZYX matrix, the extraction formulas, and the Euler-rate matrix E with $\det E = \cos\theta$ (§§2.3.2–2.3.5).
4. Explain gimbal lock geometrically (coinciding axes) and algebraically (only $\phi \mp \psi$ survives) (§2.3.4).
5. Derive Rodrigues' formula from the parallel/perpendicular split, and show it equals $e^{[\hat{n}]_\times\theta}$ (§§2.4.4–2.4.5).
6. Convert between axis–angle, rotation vector, matrix and quaternion in both directions, including the numerical special cases (§2.4.7, §2.5.5).
7. Explain why the quaternion has $\theta/2$ and why $q \equiv -q$ (§2.5.3, §2.5.6).
8. Compose and invert homogeneous transforms; project a world point into a camera (§2.6.2).
9. Explain why $SO(3)$ and $SE(3)$ are groups but not vector spaces, and what $\mathfrak{so}(3)$, $\mathfrak{se}(3)$ represent (tangent spaces of velocities and small corrections) (§2.2.4, §2.4.6, §2.6.5).
10. Derive the exponential of a twist and identify pure rotation, pure translation and a general screw (§2.6.6).
11. Use the adjoint to move a twist between frames, and the PoE formula for forward kinematics (§§2.6.7–2.6.8).

2.7.5 Practice problems (answers below)

P1. Compute $R_x(90^\circ)R_y(90^\circ)$ and $R_y(90^\circ)R_x(90^\circ)$. Where does each send e_x?

P2. Orientation 1 is 45° about z; orientation 2 is 45° about x. What single rotation angle separates them? (Use quaternions.)

P3. A revolute joint has axis $\hat{n} = e_z$ passing through $(1,2,0)$. Write its unit twist (v,ω).

P4. Find the axis and angle of $R = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$.

P5. Find the quaternion of the same R, and identify it as a product of two 90° elementary rotations.

P6. Extract ZYX Euler angles from the same R. What goes wrong with the generic formulas, and what is the correct answer?

P7. Invert $T = \begin{bmatrix} R_z(90^\circ) & (1,0,0)^\top \\ 0 & 1 \end{bmatrix}$.

P8. An IMU is mounted at $r = (0.1, 0, 0)$ m from the body origin. The body spins in place at $\omega = (0, 0, 2)$ rad/s. What velocity and acceleration does the IMU's location experience, and why does this matter for VIO?

Answers.

P1. $R_x R_y = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$ sends $e_x \mapsto e_y$; $R_y R_x = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & -1 \\ -1 & 0 & 0 \end{bmatrix}$ sends $e_x \mapsto -e_z$. Different, as expected.

P2. $q_1 \cdot q_2 = \cos^2 22.5^\circ = 0.8536$, so the angle is $2\arccos(0.8536) = 62.8^\circ$ (same as $\|\operatorname{Log}(R_1^\top R_2)\|$). Not $45^\circ + 45^\circ$, not $45^\circ\sqrt{2}$.

P3. $v = -\hat{n} \times q = -(e_z \times (1,2,0)) = -(-2,1,0) = (2,-1,0)$, $\omega = (0,0,1)$. Check: the origin, at distance $\sqrt{5}$ from the axis, moves perpendicular to the lever arm with speed $\sqrt{5}$.

P4. $\operatorname{tr} R = 0 \Rightarrow \theta = \arccos(-\frac{1}{2}) = 120^\circ$. Skew part: $(r_{32} - r_{23},\, r_{13} - r_{31},\, r_{21} - r_{12}) = (1,-1,1)$, divided by $2\sin 120^\circ = \sqrt{3}$: axis $(1,-1,1)/\sqrt{3}$.

P5. $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R} = \frac{1}{2}$ and $(x,y,z) = (r_{32} - r_{23},\, r_{13} - r_{31},\, r_{21} - r_{12})/(4w) = \frac{1}{2}(1,-1,1)$, so $q = \frac{1}{2}(1,1,-1,1) = q_x \otimes q_z$, i.e. $R = R_x(90^\circ) R_z(90^\circ)$, the second box sequence of Fig. 2.2.

P6. $r_{31} = 1 \Rightarrow \theta = -90^\circ$: gimbal lock. The generic formulas give $\operatorname{atan2}(0,0)$ for ψ and ϕ; most libraries return 0, i.e. $(0, -90^\circ, 0)$, which reconstructs a *different* matrix. Correct: only $\phi + \psi = \operatorname{atan2}(-r_{12}, r_{22}) = 90^\circ$ is determined, e.g. $(\psi, \theta, \phi) = (0, -90^\circ, 90^\circ)$ or $(30^\circ, -90^\circ, 60^\circ)$; both reproduce R exactly.

P7. $R^\top = R_z(-90^\circ)$, $-R^\top t = -(0,-1,0) = (0,1,0)$: $T^{-1} = \begin{bmatrix} R_z(-90^\circ) & (0,1,0)^\top \\ 0 & 1 \end{bmatrix}$. Check: T^{-1} maps T's origin $(1,0,0)$ to 0.

P8. $v = \omega \times r = (0, 0.2, 0)$ m/s and centripetal acceleration $\omega \times (\omega \times r) = (-0.4, 0, 0)$ m/s^2, even though the body origin is stationary. An accelerometer that is not at the body origin measures these lever-arm terms; a VIO filter must include the IMU–body extrinsic T_{BI} (and, through the adjoint, these $\omega \times r$ terms) or it will interpret rotation as translation.

2.7.6 The essence, in five lines

1. A rotation matrix is "the body axes written in reference coordinates"; everything else about $SO(3)$ follows from that.
2. Rotations live on a curved 3-D space: they compose by multiplication, never by addition.
3. Every rotation is a spin about one axis, so $R = \operatorname{Exp}(\theta\hat{n})$; the flat space of $\theta\hat{n}$ (tangent space, $\mathfrak{so}(3)$) is where you add, average, differentiate and optimise.
4. Quaternions are the double-cover encoding of the same exponential: half angles because they act from both sides, $\pm q$ because the cover is double.
5. Add translation and the same structure repeats: $T \in SE(3)$, twists in $\mathfrak{se}(3)$, Exp gives screw motions, the adjoint changes frames, and products of exponentials give robot kinematics.